第 28 章：A More Realistic Example（一个更现实的例子）

> 原书：《Learning Python》（6th Edition），作者 Mark Lutz 本章地位：上一章刚讲完 class 语法，本章立刻把它们用在"真实世界"里——通过逐步构建一个员工数据库（Person 与 Manager 类），完整演示 Python OOP 的三大特性（封装 encapsulation、继承 inheritance、多态 polymorphism）、运算符重载（operator overloading）、类与模块的关系，最后用 shelve 对象数据库实现对象持久化（object persistence）。这是一堂"跟着做"的实战课，为第 29~31 章的类机制深挖打下感性基础。

开篇引言

英文原文

We'll dig into more class syntax details in the next chapter. Before we do, though, let's take a short detour to explore a realistic example of classes in action that's more practical than what we've seen so far. In this chapter, we're going to build a set of classes that do something concrete—recording and processing infor

mation about people. As you'll see, what we call instances and classes in Python programming can often serve the same roles as records and programs in more traditional terms. The main difference here is the customization that inheritance will enable. Specifically, in this chapter we're going to code two classes: Person—a class that creates and processes information about people; Manager—a customization of Person that modifies inherited behavior. Along the way, we'll make instances of both classes and test out their functionality. When we're done, this chapter will also show you a nice example use case for classes—we'll store our instances in a simple object-oriented database, to make them permanent. That way, you can use this code as a template for fleshing out a full-blown personal database of your own written entirely in Python. Besides actual utility, though, our aim here is also educational: this chapter provides a tutorial on object-oriented programming in Python. Often, people grasp the last chapter's class syntax in the abstract but have trouble seeing how to get started when confronted with coding a new class from scratch. Toward this end, we'll take it one step at a time here, to help you learn the basics; we'll build up the classes gradually, so you can see how their features come together in complete programs. In the end, our classes will still be relatively small in terms of code, but they will demonstrate all of the main ideas in Python's OOP model. Despite it

s syntax details, Python's class system really is largely just a matter of searching for an attribute in a tree of objects, along with a special first argument for functions.

中文翻译

我们会在下一章深入探讨更多 class 语法细节。但在那之前，让我们先绕个小弯，探索一个比目前所见更实用的、类（class）实际运作的现实例子。在本章中，我们将构建一组做具体事情的类——记录和处理关于人的信息。正如你将看到的，Python 编程中所谓的 instances（实例）和 classes（类），常常可以扮演传统意义上 records（记录）和 programs（程序）的角色。这里的主要区别在于继承（inheritance）所带来的定制能力（customization）。具体来说，本章我们将编写两个类：Person——一个创建并处理人的信息的类；Manager——对 Person 的定制版本，用于修改继承来的行为。一路上，我们会创建这两个类的实例并测试其功能。完成后，本章还会向你展示一个很好的类的应用场景——我们将把实例存储在一个简单的面向对象数据库（object-oriented database）中，让它们永久保存。这样一来，你就可以把这套代码当作模板，用纯 Python 打造一个功能完整、属于你自己的个人数据库。不过，除了实际用途之外，我们这里的目标也是教育性的（educational）：本章提供了一堂 Python 面向对象编程（OOP）的实践教程。通常，人们能抽象地理解上一章的 class 语法，但当面对从零编写一个新类时，却不知道该如何下手。为此，我们将在这里一步一步来，帮助你掌握基础；我们会逐步构建这些类，让你看到它们的特性是如何在完整程序中结

合在一起的。最终，我们的类从代码量上说仍然相对较小，但它们将演示 Python OOP 模型中所有的主要思想。尽管语法细节繁多，Python 的类系统实际上在很大程度上就是"在一棵对象树中查找属性，外加函数的一个特殊首参数"而已。

深度理解

·核心概念：本章是 OOP 的"实操总动员"。作者明确告诉你 OOP 的本质——继承带来的定制能力是 Python 的实例/类与传统记录/程序的最大区别。先记住这句话：类是"对象树中的属性查找"（attribute search in a tree of objects）加上方法函数的特殊首参数 self。

·设计原因：为什么要先做这个案例再深入语法？因为作者教学经验表明："语法懂了、动手不会"是新手最常见的卡点。先建立"类的完整生命周期"的整体图景，后面的语法细节才有安放之处。

·生活类比：把 Person 类想象成一张"员工信息表模板"，把 Manager 类想象成模板的"加注版本"——表格结构相同，但"涨薪"这一栏的算法不一样。每个实例（bob、sue）就是填好的一行记录。

·初学者误区：以为"类"很神秘。作者提醒：类本质上只是命名空间 + 继承查找，self 只是 Python 自动传入实例的特殊首参数。越早戳破这层窗户纸，后面越轻松。

·本章路线图：7 个 Step：建实例 → 加行为方法 → 运算符重载 → 子类定制 → 定制构造函数 → 内省工具 → 存入数据库。

Step 1: Making Instances（第一步：创建实例）

英文原文

> OK, so much for the design phase—let's move on to implementation. Our first task is to start coding the main class, Person. In your favorite text editor, open a new file for the code we'll be writing. As noted in the prior chapter, it's a fairly strong convention in Python to begin module names with a lowercase letter and class names with an uppercase letter. Like the name of self arguments in methods, this is not required by the language, but it's so common that deviating might be confusing to people who later read your code. To conform, we'll call our new module file person.py and our class within it Person, as in Example 28-1.

Example 28-1: person1.py（开始）

```
class Person:   #
```

中文翻译

好了，设计阶段到此为止——让我们进入实现阶段。第一个任务是开始编写主类 Person。在你喜欢的文本编辑器中，为我们将要编写的代码打开一个新文件。正如前一章所述，Python 中有一个相当强烈的约定：模块名以小写字母开头，类名以大写字母开头。就像方法中 self 参数的名称一

样，语言并不强制要求，但它太常见了，偏离它可能会让后来阅读你代码的人感到困惑。为了遵守约定，我们将新模块文件命名为 person.py，其中的类命名为 Person，如 Example 28-1 所示。我们会在前进过程中不断修改这个文件。为了帮你跟踪它的各个版本，我们会在标题中给文件名加上示例编号（example number），并且既在示例包中使用这个编号，也在这里的运行和导入命令中使用。每次修订的新改动都会用粗体显示。意图是展示对单个文件的修改，但书是线性的。正如前一章所述，在 Python 中我们可以往单个模块文件里写任意数量的函数和类，而如果以后往里面添加无关组件，person.py 这个名字就可能不太合理了。目前我们先假设其中的一切都与 Person 相关。反正本来就该如此——正如我们所学的，模块在具有单一、内聚的（cohesive）用途时往往工作得最好。

深度理解

·核心概念：命名约定（naming convention）与模块内聚性（cohesion）。person.py（小写）是模块，Person（大写）是类——这是社区约定而非语法要求；而"一个模块只干一件事"是模块设计的第一原则。

·底层实现：模块文件是一个命名空间（namespace），类名只是模块命名空间里的一个变量名——所以 import person 之后要写 person.Person 才能访问。类名可以像任何变量一样被赋值、被传递。

·设计原因：为什么模块小写、类大写？因为代码被阅读的次数远多于被编写的次数；一眼区分"这是模块还是类"能降低认知负担。

·实际问题：命名约定解决的是"代码考古"问题——半年后重读代码，from person import Person 这种写法让人立刻知道层级关系；内聚的模块则保证"改一处不炸全局"。

·初学者误区：以为改名会报错。不会——Python 不管大小写，class person: 也能运行；只是会坑了读你代码的人（包括未来的你）。

Coding Constructors（编写构造函数）

英文原文

> Now, the first thing we want to do with our Person class is record basic information about people—to fill out record fields, if you will. Of course, these are known as instance object attributes in Python-speak, and they generally are created by assignment to self attributes in a class's method functions. The normal way to give instance attributes their first values is to assign them to self in the init constructor method, which contains code run automatically by Python each time an instance is created. Let's add one to our class, in Example 28-2.

Example 28-2: person2.py（添加属性初始化）

```
class Person:
    def __init__(self, name, job, pay):     #
        self.name = name                     #
        self.job = job                       # self
        self.pay = pay
```

This is a very common coding pattern: we pass in the data to be attached to an instance as arguments to t he constructor method and assign them to self to ret ain them permanently. In OO terms, self is the newly created instance object, and name, job, and pay beco me state information—descriptive data saved on an object for later use. Although other techniques (such as enclosing scope reference closures) can save detail s, too, instance attributes make this very explicit and easy to understand. Notice that the argument names appear twice here. This code might even seem a bit r edundant at first, but it's not. The job argument, for e xample, is a local variable in the scope of the init fun ction, but self.job is an attribute of the instance that's the implied subject of the method call. They are two different variables, which happen to have the same n ame. By assigning the job local to the self.job attribut e with self.job=job, we save the passed-in job on the instance for later use. As usual in Python, where a na me is assigned, or what object it is assigned to, deter mines what it means. Speaking of arguments, there's really nothing magical about init, apart from the fact that it's called automatically when an instance is mad e, and has a special first argument. Despite its weird

name, it's a normal function and supports all the features of functions we've already covered. We can, for example, provide defaults for some of its arguments, so they need not be provided in cases where their values aren't available or useful. To demonstrate, let's make the job argument optional—it will default to None, meaning the person being created is not (currently?) employed. If job defaults to None, we'll probably want to default pay to 0, too, for consistency (unless some of the people you know manage to get paid without having jobs). In fact, we have to specify a default for pay because according to Python's syntax rules and Chapter 18, any arguments in a function's header after the first default must all have defaults, too. Example 28-3 codes the mod.

Example 28-3: person3.py（添加构造函数默认值）

```
class Person:
    def __init__(self, name, job=None, pay=0):   #
        self.name = name
        self.job = job
        self.pay = pay
```

What this code means is that we'll need to pass in a name when making Persons, but job and pay are now optional; they'll default to None and 0 if omitted. The self argument, as usual, is filled in by Python automatically to refer to the instance object—assigning values to attributes of self attaches them to the new instance.

中文翻译

现在，我们想对 Person 类做的第一件事是记录人的基本信息——姑且称之为填写记录字段。当然，在 Python 术语中这叫做实例对象的属性（attributes），它们通常通过给类的方法函数中的 self 属性赋值来创建。给实例属性赋予初始值的常规做法，是在 init 构造方法（constructor method）中把它们赋值给 self——这个方法包含的代码会在每次创建实例时由 Python 自动运行。让我们把这个方法加到类中，如 Example 28-2 所示。这是一个非常常见的编码模式：我们把要附加到实例上的数据作为参数传给构造方法，然后把它们赋值给 self 以永久保留。用面向对象（OO）的术语说，self 就是新创建的实例对象，name、job、pay 成为状态信息（state information）——保存到对象上供日后使用的描述性数据。虽然其他技术（比如外层作用域引用闭包 enclosing scope reference closures）也能保存数据，但实例属性让这一切非常显式、易于理解。请注意，这里的参数名出现了两次。这段代码乍看起来可能有点冗余，其实不然。例如，job 参数是 init 函数作用域内的一个局部变量，而 self.job 是实例的属性——实例是方法调用的隐含主体（implied subject）。它们是两个恰好同名的不同变量。通过 self.job = job 把局部变量 job 赋给属性 self.job，我们就把传入的 job 保存到实例上供日后使用。和 Python 中一贯的规则一样，一个名字在哪里被赋值、被赋到哪个对象上，决定了它的含义。说到参数，init 其实毫无神秘之处——除了它会在创建实例时被自动调用、且有一个特殊首参数之外。尽管名字怪异，它就是一个普通函数，支持我们已经学过的函数的所有特性。例如，我们可以为它

的某些参数提供默认值（defaults），这样在值不可用或没意义时就不必提供。为了演示，我们让 job 参数变成可选的——默认值为 None，表示正在创建的人（目前？）没有工作。如果 job 默认为 None，出于一致性考虑，我们可能也想让 pay 默认是 0（除非你认识的某些人不需要工作也能领到钱）。事实上，我们必须为 pay 指定默认值，因为根据 Python 的语法规则和第 18 章的内容，函数头中第一个默认参数之后的任何参数也都必须有默认值。Example 28-3 实现了这一修改。这段代码的意思是：创建 Person 时必须传入 name，但 job 和 pay 现在是可选的；如果省略，它们分别默认为 None 和 0。self 参数一如既往由 Python 自动填充，指向实例对象——给 self 的属性赋值，就是把它们挂到新实例上。

深度理解

·核心概念：init 是 Python 的构造方法——严格说是"初始化方法"（initializer）：对象的内存其实已由底层分配好，init 负责给新对象填充属性。self 就是 Python 自动传入的那个"新实例"。

·底层实现：调用 Person('Bob') 时，解释器大致分两步走：先由 type（元类）创建新实例对象，再自动调用 Person.init(新实例,'Bob')。所以 self.name = name 的本质是"给刚出生的实例挂上第一个属性"。属性最终存放在实例的 dict 字典里。

·设计原因：为什么参数名出现两次（self.job = job）？因为 job 是函数局部变量，self.job 是实例属性——"赋值给谁"决定含义。这是 Python 极简设计的体现：没有 t

his->、没有显式属性声明，赋值语句本身就是在创建属性。

·实际问题：默认参数（default argument）解决"可选的字段"问题——不是每个人都有工作和薪水；job=None表达"（目前）无业"这一状态。

·初学者误区：① 以为 init 是"分配内存"的——不是，它只是初始化；② 忘记 self. 前缀，导致数据只存在局部变量里，实例上什么都没有——这是新手最常见的 bug 之一。

代码分析

```
class Person:
    def __init__(self, name, job=None, pay=0):   # …
        self.name = name      #  name  __dict__ 'name…
        self.job = job        #  'job'
        self.pay = pay        #  'pay'
```

·解释器如何处理：执行 bob = Person('Bob Smith') 时，Python 先为 bob 分配实例对象，然后调用 Person.init(bob,'Bob Smith')——job 与 pay 使用默认值 None、0。self.name = name 这行赋值，解释器会在实例的 dict 字典里写入 {'name':'Bob Smith','job': None,'pay': 0}。

·设计思想：构造方法的职责单一——"把传入数据变成实例状态"。把所有字段初始化集中在一处，实例一出生就结构完整，后续任何方法都可以放心读取。

Testing as You Go（边写边测）

英文原文

> This class doesn't do much yet—it essentially just fills out the fields of a new record—but it's a real working class. At this point, we could add more code to it for more features, but we won't do that yet. As you've pr obably begun to appreciate already, programming in Python is really a matter of incremental prototyping —you write some code, test it, write more code, test again, and so on. Because Python provides both an in teractive session and nearly immediate turnaround af ter code changes, it's more natural to test as you go t han to write a huge amount of code to test all at onc e. Before adding more features, then, let's test what we've got so far by making a few instances of our cla ss and displaying their attributes as created by the co nstructor. We could do this interactively, but as you'v e also probably surmised by now, interactive testing has its limits—it gets tedious to have to reimport mo dules and retype test cases each time you start a new testing session. More commonly, Python programme rs use the interactive prompt for simple one-off tests but do more substantial testing by writing code at th e bottom of the file that contains the objects to be te sted, as in Example 28-4.

Example 28-4: person4.py（添加增量式自测代码）

```
class Person:
    def __init__(self, name, job=None, pay=0):
        self.name = name
        self.job = job
        self.pay = pay

bob = Person('Bob Smith')                          #
sue = Person('Sue Jones', job='dev', pay=100000) #  __init__
print(bob.name, bob.pay)                           #
print(sue.name, sue.pay)                           # sue  bob
```

Notice here that the bob object accepts the defaults for job and pay, but sue provides values explicitly. Als o, note how we use keyword arguments when makin g sue; we could pass by position instead, but the key words may help remind us later what the data is, and they allow us to pass the arguments in any left-to-rig ht order we like. Again, despite its unusual name, init is a normal function, supporting everything you alrea dy know about functions—including both defaults an d pass-by-name keyword arguments. When this file r uns as a script, the test code at the bottom makes tw o instances of our class and prints two attributes of e ach—name and pay: $ python3 person4.py Bob Smit h 0 Sue Jones 100000 You can also type this file's tes t code at Python's interactive prompt (assuming you i mport the Person class there first), but coding canne d tests inside the module file like this makes it much easier to rerun them in the future. Although this is fai rly simple code, it's already demonstrating somethin

g important. Notice that bob's name is not sue's, and sue's pay is not bob's. Each is an independent record of information. Technically, bob and sue are both namespace objects—like all class instances, they each have their own independent copy of the state information created by the class. Because each instance of a class has its own set of self attributes, classes are a natural for recording information for multiple objects this way; just like built-in types such as lists and dictionaries, classes serve as a sort of object factory. Other Python program structures, such as functions and modules, have no such concept. Chapter 17's closure functions come close in terms of per-call state but don't have the multiple methods, inheritance, and larger structure we get from classes.

中文翻译

这个类现在还做不了多少事——它基本上只是填好一条新记录的字段——但它已经是一个真正能工作的类了。此时，我们本可以为它添加更多代码来实现更多功能，但我们暂时还不会那样做。你可能已经开始体会到：Python 编程实际上就是一个增量式原型开发（incremental prototyping）的过程——写一点代码，测试，再写一点，再测试，如此往复。因为 Python 既提供交互式会话，代码改动后又能几乎立即看到结果，所以边写边测比一口气写完一大堆代码再统一测试要自然得多。因此，在添加更多功能之前，让我们先测试一下目前已有的成果——创建类的几个实例，并显示构造函数为它们创建的属性。我们本可以在交互模式下

做这件事，但你大概也已经猜到，交互式测试有它的局限——每次开始新一轮测试会话都要重新导入模块、重新输入测试用例，实在繁琐。更常见的做法是：Python 程序员用交互式提示符做简单的一次性测试，而把更实质性的测试写在被测试对象所在文件的底部，如 Example 28-4 所示。注意这里：bob 对象接受了 job 和 pay 的默认值，而 sue 显式提供了值。另外，注意我们在创建 sue 时使用了关键字参数（keyword arguments）；我们本可以按位置传参，但关键字有助于日后提醒我们这些数据是什么，也允许我们按任意从左到右的顺序传参。再说一次，尽管名字特殊，init 就是一个普通函数，支持你已经了解的函数的全部特性——包括默认值和按名传参的关键字参数。当这个文件作为脚本运行时，底部的测试代码会创建我们类的两个实例，并打印每个实例的两个属性——name 和 pay： $ python3 person4.py Bob Smith 0 Sue Jones 100000 你也可以在 Python 交互式提示符下输入这个文件的测试代码（前提是先在那里导入 Person 类），但像这样把预设测试写在模块文件内部，以后重跑它们就容易得多。虽然这只是相当简单的代码，但它已经在演示一件重要的事情。注意 bob 的 name 不是 sue 的，sue 的 pay 也不是 bob 的。每一个都是独立的信息记录。从技术上讲，bob 和 sue 都是命名空间对象（namespace objects）——和所有类实例一样，它们各自拥有一份由类创建的、独立的状态信息副本。因为类的每个实例都有自己的 self 属性集合，类天然适合以这种方式为多个对象记录信息；就像列表和字典等内置类型一样，类充当一种对象工厂（object factory）。Python 的其他程序结构，比如函数和模块，没有这样的概念。第 17 章的闭包函数在"每次调用的状态"方面很接近，但没有类提供的

多方法、继承和更大的结构。

深度理解

·核心概念：增量式原型开发（incremental prototyping）是 Python 的文化：小步快跑、边写边测。同时理解"实例是独立的命名空间对象"——bob 和 sue 的 dict 互不相干。

·底层实现：每个实例内部有一个 dict 字典，self.name = name 就是往这个字典里塞键值对。bob 与 sue 是两个不同的字典，所以数据互不干扰；类则像一个"工厂"，负责生产这些带默认结构的对象。

·设计原因：为什么把测试代码写在模块底部而不是交互模式？因为可重跑性——交互式会话每次都要重新输入；写在文件底部则 python3 person4.py 一条命令搞定。这为后面 name =='main' 的引入埋下伏笔。

·实际问题：关键字参数 job='dev', pay=100000 解决"参数语义易忘"的问题——代码即文档；同时允许任意顺序传参。

·初学者误区：以为"类实例"和"类"是同一个东西。其实：类是模板（一个对象），实例是模板造出来的独立产品（多个对象）。另一个误区：以为闭包能替代类——作者明确说闭包没有"多方法 + 继承"的大结构。

代码分析

```
class Person:
    def __init__(self, name, job=None, pay=0):
        self.name = name
        self.job = job
        self.pay = pay

bob = Person('Bob Smith')                          # job/pay …
sue = Person('Sue Jones', job='dev', pay=100000) #  __init__…
print(bob.name, bob.pay)                           #  __di…
print(sue.name, sue.pay)                           # sue  bob
```

·解释器如何处理：Person('Bob Smith') 触发 init，把'Bob Smith'、None、0 分别写入 bob 的 dict。sue = Person('Sue Jones', job='dev', pay=100000) 同理，但用的是关键字传参。print(bob.name, bob.pay) 时，解释器先查 bob 的 dict，取到'Bob Smith' 和 0。整个过程不涉及任何"复制类"——两个实例只是恰好由同一个类造出来。

·设计思想："类即工厂"：同一套构造逻辑，产出多份互不干扰的状态。这正是记录型数据（record）最朴素的实现方式。

Using Code Two Ways（一份代码两种用法）

英文原文

> As is, the test code at the bottom of the file works, but there's a big catch—its top-level print statements run both when the file is run as a script and when it is imported as a module. This means if we ever decide to import the class in this file in order to use it some

where else (and we will soon in this chapter), we'll see the output of its test code every time the file is imported. That's not very good software citizenship, though: client programs probably don't care about our internal tests and won't want to see our output mixed in with their own. Although we could split the test code off into a separate file, it's often more convenient to code tests in the same file as the items to be tested. It would be better to arrange to run the test statements at the bottom only when the file is run for testing, not when the file is imported. As linear readers of this book have already learned, that's exactly what the module name check is designed for. Example 28-5 shows what this addition looks like—simply add the required test and indent your self-test code.

Example 28-5: person5.py（同时支持导入和运行/测试）

```
class Person:
    def __init__(self, name, job=None, pay=0):
        self.name = name
        self.job = job
        self.pay = pay

if __name__ == '__main__':          #
    bob = Person('Bob Smith')
    sue = Person('Sue Jones', job='dev', pay=100000)
    print(bob.name, bob.pay)
    print(sue.name, sue.pay)
```

Now, we get exactly the behavior we're after—runnin

g the file as a top-level script tests it because its nam e is main, but importing it as a library of classes later does not: $ python3 person5.py Bob Smith 0 Sue Jon es 100000 $ python3 >>> import person5 >>> Whe n imported, the file now defines the class but does n ot use it. When run directly, this file creates two insta nces of our class as before, and prints two attributes of each; again, because each instance is an independ ent namespace object, the values of their attributes d iffer.

中文翻译

目前，文件底部的测试代码可以工作，但有一个大问题——它的顶层 print 语句在文件作为脚本运行时会执行，在作为模块被导入时也会执行。这意味着如果我们决定导入这个文件里的类、在别处使用它（本章很快就要这么做了），那么每次导入该文件时都会看到测试代码的输出。这不是很好的"软件公民"行为：客户程序大概不在乎我们的内部测试，也不希望看到自己的输出被我们的输出混入。虽然我们可以把测试代码拆到单独的文件里，但把测试和被测对象写在同一个文件里通常更方便。更好的做法是：让底部的测试语句仅在文件为测试而运行时执行，而导入时不执行。线性阅读本书的读者已经学过，这正是模块的 name 检查的用途。E xample 28-5 展示了这个改动——只需加上必要的判断并缩进你的自测代码。现在，我们得到了想要的行为——作为顶层脚本运行时文件会测试自己，因为它的 name 是 ma in；而之后作为类库被导入时不会执行测试： $ python3 p erson5.py Bob Smith 0 Sue Jones 100000 $ python3

>>> import person5 >>> 被导入时，这个文件现在只定义类而不使用它。直接运行时，这个文件会像之前一样创建类的两个实例，并打印每个实例的两个属性；同样，因为每个实例都是独立的命名空间对象，它们的属性值各不相同。

深度理解

·核心概念：if name =='main': 是 Python 模块的"双重身份"机制——一个 .py 文件既可以当脚本直接运行，也可以当库被导入，靠 name 区分当前身份。

·底层实现：Python 启动执行脚本时，会把该模块的 name 变量设为字符串'main'；被 import 时则设为模块的真实名字（如'person5'）。判断只是普通的字符串比较，背后没有魔法。

·设计原因：为什么不用两个文件？因为就近维护——测试与被测代码放一起，改动时不会忘；同时又不污染导入方（"良好的软件公民"）。这是 Python 生态最普遍的测试入门形态。

·实际问题：它让"模块既是被复用的库，又是可独立演示的程序"成为可能——这也是为什么很多 Python 库的主程序都包在这个判断里。

·初学者误区：① 忘写这个判断，导入库时看到一堆测试输出；② 以为 name 是"文件名"——它是模块名，不是 file。③ 以为每个模块都有这个判断才会被导入——不写也能导入，只是导入时会执行顶层代码。

代码分析

```
class Person:
    def __init__(self, name, job=None, pay=0):
        self.name = name
        self.job = job
        self.pay = pay

if __name__ == '__main__':          # __name__  '__main_…
    bob = Person('Bob Smith')       #
    sue = Person('Sue Jones', job='dev', pay=100000)  #
    print(bob.name, bob.pay)        #  bob
    print(sue.name, sue.pay)        #  sue
```

·解释器如何处理：解释器先编译并执行整个文件的顶层代码——定义 Person 类（类对象被创建并绑定到模块命名空间）。接着判断 name：直接运行时值为'main'，条件成立，执行测试体；被 import person5 时，name 为'person5'，条件不成立，测试体被跳过——但类仍已定义好，可供导入方使用。

·设计思想：这份代码展示了模块的双重用途——"可执行程序"与"可导入库"二合一。它是 Python"测试与被测同文件"文化的基石。

Step 2: Adding Behavior Methods（第二步：添加行为方法）

英文原文

Everything looks good so far—at this point, our class is essentially a record factory; it creates and fills out f

ields of records (attributes of instances, in Pythonic t erms). Even as limited as it is, though, we can still run some operations on its objects. Although classes add an extra layer of structure, they ultimately do most of their work by embedding and processing core object types like lists and strings. In other words, if you alre ady know how to use Python's simple core objects, y ou already know much of the Python class story; clas ses are really just a minor structural extension. For ex ample, the name field of our objects is a simple strin g, so we can extract last names from our objects by s plitting on spaces and indexing. These are all core-ob ject operations, which work whether their subjects ar e embedded in class instances or not:

```
 >>> name ='Bob Smith' # Simple string, outside cla
ss >>> name.split() # Extract last name ['Bob','Smith'
]

>>> name.split()[-1] # Or [1], if always just two parts'
Smith'
```

Similarly, we can give an object a pay raise by updati ng its pay field—that is, by changing its state informa tion in place with an assignment. This task also involv es basic operations that work on Python's core object s, regardless of whether they are standalone or embe dded in a class structure (formatting here masks any extraneous digits):

>>> pay = 100000 # Simple variable, outside class >>> pay = 1.10 # Give a 10% raise >>> print(f'{pay:,.2f}') # Or: pay = pay 1.10, if you like to type 110,000.00 # Or: pay = pay + (pay .10), if you really do

To apply these operations to the Person objects created by our script, simply do to bob.name and sue.pay what we just did to name and pay, as listed in Example 28-6. The operations are the same, but the subjects are attached as attributes to objects created from our class.

Example 28-6: person6.py（处理内嵌的内置对象）

```
class Person:
    def __init__(self, name, job=None, pay=0):
        self.name = name
        self.job = job
        self.pay = pay
if __name__ == '__main__':
    bob = Person('Bob Smith')
    sue = Person('Sue Jones', job='dev', pay=100000)
    print(bob.name, bob.pay)
    print(sue.name, sue.pay)
    print(bob.name.split()[-1])     #
    sue.pay *= 1.10                 #
    print(f'{sue.pay:,.2f}')
```

We've added the last three lines here; when they're run, we extract bob's last name by using basic string and list operations on his name field, and give sue a pay raise by modifying her pay attribute in place with basic number operations. In a sense, sue is also a mutable object—her state changes in place just like a lis

t after an append call. Here's the new version's outpu t when run as a top-level script:

> $ python3 person6.py Bob Smith 0 Sue Jones 10000 0 Smith 110,000.00
>
> The preceding code works as planned, but if you sho w it to a veteran software developer he or she will pr obably tell you that its general approach is not a gre at idea in practice. Hardcoding operations like these outside of the class can lead to maintenance problem s in the future. For example, what if you've hardcode d the last-name-extraction formula at many different places in your program? If you ever need to change t he way it works (to support a new name structure, for instance), you'll need to hunt down and update every occurrence. Similarly, if the pay-raise code ever chan ges (e.g., to require approval or database updates), y ou may have multiple copies to modify. Just finding a ll the appearances of such code may be problematic i n larger programs—they may be scattered across ma ny files and folders, split into individual steps, and so on. In a prototype like this, frequent change is almost guaranteed.

中文翻译

到目前为止一切顺利——此时我们的类本质上是一个记录工厂（record factory）：它创建记录并填好字段（用 Pyt hon 的术语说，就是实例的属性）。尽管它还这么简陋，我们仍然可以对它的对象执行一些操作。虽然类增加了一层

结构，但它们最终大部分工作还是靠内嵌并处理核心对象类型（core object types）——比如列表和字符串——来完成的。换句话说，如果你已经会使用 Python 简单的核心对象，你就已经了解了 Python 类的多半故事；类其实只是一种很小的结构性扩展。例如，我们对象的 name 字段就是一个简单的字符串，所以我们可以通过按空格拆分（split）再索引（indexing）来从对象中提取姓氏。这些都是核心对象操作，无论它们的操作对象是否被嵌入在类实例中，都照常工作： >>> name ='Bob Smith' # 类外面的简单字符串>>> name.split() # 提取姓氏['Bob','Smith'] >>> name.split()[-1] # 或者用 [1]，如果永远只有两部分'Smith' 类似地，我们可以通过更新对象的 pay 字段来给它涨薪——也就是说，用一次赋值就地（in place）修改它的状态信息。这个任务同样只用对 Python 核心对象的基本操作，无论对象是独立存在还是嵌入在类结构中（这里用格式化把多余的位数遮掉了）： >>> pay = 100000 # 类外面的简单变量>>> pay = 1.10 # 涨薪 10% >>> print(f'{pay:,.2f}') # 或者：pay = pay 1.10，如果你喜欢多打字110,000.00 # 或者：pay = pay + (pay .10)，如果你真的喜欢要把这些操作应用到我们脚本创建的 Person 对象上，只需对 bob.name 和 sue.pay 做我们刚刚对 name 和 pay 做的事，如 Example 28-6 所示。操作是一样的，只是操作对象变成了挂在类创建的对象上的属性。我们在这里添加了最后三行；运行时，我们用基本的字符串和列表操作从 bob 的姓名字段中提取姓氏，并用基本的数字操作就地修改 sue 的 pay 属性给她涨薪。从某种意义上说，sue 也是一个可变（mutable）对象——她的状态就地改变，就像列表在 append 调用后那样。以下是新版本作为顶层脚本运行时的输出

： $ python3 person6.py Bob Smith 0 Sue Jones 100000 Smith 110,000.00 前面的代码按计划工作了，但如果你把它拿给资深软件开发者看，他或她大概会告诉你：这种做法在实践中不是好主意。把这样的操作硬编码（hardcoding）在类的外面，会给未来的维护带来麻烦。例如，如果程序里许多不同的地方都硬编码了姓氏提取公式会怎样？如果你需要改变它的工作方式（比如支持新的名字结构），你就得找到并更新每一处出现的地方。同样，如果涨薪代码将来有变化（例如需要审批或数据库更新），你可能有多份副本要改。在大程序中，光是找到这些代码的所有出现位置可能就很麻烦——它们可能散落在许多文件和文件夹里、被拆成一个个步骤，等等。在这样的原型里，频繁的变更几乎是必然的。

深度理解

·核心概念：类是"结构外壳"，真正干活的是内嵌的核心对象（字符串、数字、列表）。bob.name 就是一个普通字符串，能用字符串的一切方法；sue.pay 就是一个普通数字，能参与一切算术。

·底层实现：属性查找先查实例的 dict（'name' →'Bob Smith'），拿到字符串对象后，.split() 是字符串类型的方法调用，与"它住在类里"毫无关系。实例的可变性来自其 dict 中保存的引用可以重新指向新对象。

·设计原因：作者强调"类只是微小结构性扩展"——降低学习曲线：会核心对象就会类。这也解释了为什么 Python 类的"数据"部分根本不用声明，直接复用内置类型。

·实际问题：硬编码（hardcoding）在类外会引发维护噩梦：公式出现 N 处，改一处漏一处。这引出下一节的封装（encapsulation）。生活类比：把"计算税后工资"的公式写在 20 个报销单上，税改时你就要改 20 张——不如把公式做成一个统一的"计算器工具"。

·初学者误区：以为类的存在意味着"不能用内置操作"——恰恰相反，类就是用来"打包"这些内置操作的。

代码分析

```
if __name__ == '__main__':
    bob = Person('Bob Smith')
    sue = Person('Sue Jones', job='dev', pay=100000)
    print(bob.name, bob.pay)        #
    print(sue.name, sue.pay)
    print(bob.name.split()[-1])     #
    sue.pay *= 1.10                 #
    print(f'{sue.pay:,.2f}')        # f-string  +
```

·解释器如何处理：bob.name.split()[-1] 分三步求值：取 bob.name（字符串）→ 调用 .split()（返回列表）→ 索引 [-1]（取最后一个元素，即姓）。sue.pay = 1.10 是 sue.pay = sue.pay 1.10 的简写：读属性（整数 100000）→ 与 1.10 相乘（得浮点数）→ 赋值回 sue.dict['pay']。f-string 中的 :,.2f 是格式说明符——逗号千分位、两位小数。

·设计思想：数据与操作分离是"过程式"写法的自然结果；但本节末尾作者揭示了它的致命伤——重复。这正是下一步要引入方法的动机。

Coding Methods（编写方法）

英文原文

> What we really want to do here is employ a software design concept known as encapsulation—wrapping up operation logic behind interfaces, such that each operation is coded only once in our program. That way, if our needs change in the future, there is just one copy to update. Moreover, we're free to change the single copy's internals almost arbitrarily, without breaking the code that uses it. In Python terms, we want to code operations on objects in a class's methods, instead of littering them throughout our program. In fact, this is one of the things that classes are very good at—factoring code to remove redundancy and thus optimize maintainability. As an added bonus, turning operations into methods enables them to be applied to any instance of the class, not just those that they've been hardcoded to process. This is all simpler in code than it may sound in theory. Example 28-7 achieves encapsulation by moving the two operations from code outside the class to methods inside the class. While we're at it, let's change our self-test code at the bottom to use the new methods we're creating, instead of hardcoding operations.

Example 28-7: person7.py（添加方法以封装操作）

```
class Person:
    def __init__(self, name, job=None, pay=0):
        self.name = name
        self.job = job
        self.pay = pay
    def lastName(self):                          #
        return self.name.split()[-1]             # self
    def giveRaise(self, percent):
        self.pay = int(self.pay * (1 + percent))  #
if __name__ == '__main__':
    bob = Person('Bob Smith')
    sue = Person('Sue Jones', job='dev', pay=100000)
    print(bob.name, bob.pay)
    print(sue.name, sue.pay)
    print(bob.lastName(), sue.lastName())       #
    sue.giveRaise(.10)                          #
    print(sue.pay)
```

As we've learned, methods are simply normal functio ns that are attached to classes and designed to proce ss instances of those classes. The instance is the subj ect of the method call and is passed to the method's self argument automatically. The transformation to th e methods in this version is straightforward. The new lastName method, for example, simply does for self what the previous version hardcoded for bob, becaus e self is the implied subject when the method is calle d. lastName also returns the result because this oper ation is a called function now; it computes a value for its caller to use arbitrarily, even if it is just to be print ed. Similarly, the new giveRaise method just does for

self what we did for sue before. When run now, our file's output is similar to before—we've mostly just refactored the code to allow for easier changes in the future (command lines that run the most recent mod are often omitted from here on for space):

```
Bob Smith 0 Sue Jones 100000 Smith Jones 110000
```

A few coding details are worth pointing out here. First, notice that sue's pay is now still an integer after a pay raise—we convert the math result back to an integer by calling the int built-in within the method. Changing the value to either int or float is probably not a significant concern for this demo: integer and floating-point objects have the same interfaces and can be mixed within expressions. Still, we may need to address truncation and rounding issues in a real system—money probably is significant to Persons! As covered in Chapter 5, we might handle this by using the round(N, 2) built-in to round and retain cents, using the decimal type to fix precision, or storing monetary values as full floating-point numbers and displaying them with a formatting string to show cents as we did earlier. For now, we'll simply truncate any cents with int. Second, notice that we're also printing sue's last name this time—because the last-name logic has been encapsulated in a method, we get to use it on any instance of the class. As we've seen, Python tells a method which instance to process by automatically passing it in to the first argument, usually called self. Specif

ically: In the first call, bob.lastName(), bob is the implied subject passed to self. In the second call, sue.lastName(), sue goes to self instead. Trace through these calls to see how the instance winds up in self—it's a key concept. The net effect is that the method fetches the name of the implied subject each time. The same happens for giveRaise. We could, for example, give bob a raise by calling giveRaise for both instances this way, too. Unfortunately for bob, though, his zero starting pay will prevent him from getting a raise as the program is currently coded—nothing times anything is nothing, something we may want to address in a future 2.0 release of our software. Finally, notice that the giveRaise method assumes that percent is passed in as a floating-point number between zero and one. That may be too radical an assumption in the real world (a 1,000% raise would probably be a bug for most of us!); we'll let it pass for this prototype, but we might want to test or at least document this in a future iteration of this code. Stay tuned for a rehash of this idea in later chapters, where we'll explore function decorators and Python's assert statement as alternatives that can do the validity test for us automatically during development. In Chapter 39, for example, we'll write a tool that lets us validate with strange incantations like the following:

python @rangetest(percent=(0.0, 1.0)) # Use decorator to validate def giveRaise(self, percent): self.pay = i

nt(self.pay (1 + percent))

中文翻译

我们真正想在这里做的是运用一个名为封装（encapsulation）的软件设计概念——把操作逻辑包在接口后面，使每个操作在程序里只编写一次。这样一来，将来需求变了，只有一个副本需要更新。而且，我们几乎可以随意改动这唯一副本的内部实现，而不会破坏使用它的代码。用 Python 的话说，我们要把对对象的操作写成类里的方法（methods），而不是让它们散落在程序各处。事实上，这正是类最擅长的能力之一——分解（factoring）代码以消除冗余（redundancy），从而优化可维护性。额外的好处是：把操作变成方法后，它们就能应用于类的任何实例，而不仅仅是被硬编码指定处理的那几个。所有这些用代码表达起来比理论听起来简单得多。Example 28-7 通过把两个操作从类外代码移到类内方法中实现了封装。顺便，我们把底部的自测代码也改成使用我们正在创建的新方法，而不是硬编码操作。正如我们学过的，方法就是附着在类上、设计用来处理该类实例的普通函数。实例是方法调用的主体，会被自动传给方法的 self 参数。这个版本向方法的转换很直接。例如，新的 lastName 方法就是为 self 做上一版为 bob 硬编码的事，因为方法被调用时 self 就是隐含的主体。lastName 还返回了结果，因为这个操作现在是"被调用的函数"；它计算出一个值交给调用方随意使用——哪怕只是打印出来。同样，新的 giveRaise 方法也只是为 self 做我们之前为 sue 做的事。现在运行时，文件的输出与之前类似——我们主要是重构（refactored）了代码，让将来更容易修改（从这儿往后，为节省篇幅，运行最新版本的命令行常常省略）：

Bob Smith 0 Sue Jones 100000 Smith Jones 110000
有几处编码细节值得指出。第一，注意 sue 涨薪后的 pay 现在仍然是整数（integer）——我们在方法内部调用 int 内置函数把数学运算结果转回整数。对这个演示来说，把值改成 int 还是 float 大概不是什么大问题：整数和浮点对象接口相同，可以在表达式里混用。不过，在真实系统里我们可能需要处理截断和舍入的问题——钱对 Person 来说恐怕很重要！如第 5 章所述，我们可以用 round(N, 2) 内置函数四舍五入保留分，用 decimal 类型固定精度，或者把货币值存成完整浮点数并用格式化字符串显示分（就像我们前面做的）。眼下，我们只是用 int 截掉所有分。第二，注意这次我们也打印了 sue 的姓——因为姓氏逻辑已经被封装进方法，我们可以对类的任何实例使用它。正如我们所见，Python 通过自动把实例传入第一个参数（通常叫 self）来告诉方法要处理哪个实例。具体来说：第一次调用 bob.lastName() 时，bob 作为隐含主体被传给 self；第二次调用 sue.lastName() 时，换成 sue 进入 self。自己跟踪一遍这些调用，看看实例是如何进入 self 的——这是一个关键概念。净效果是：方法每次都取隐含主体的名字。giveRaise 也一样。例如，我们也可以用同样的方式对两个实例都调用 giveRaise 给 bob 涨薪。不过对 bob 不幸的是，按现在程序的编码，他从 0 起步的薪水注定涨不起来——任何数乘以 0 还是 0。这或许值得在我们软件未来的 2.0 版本里解决。最后，注意 giveRaise 方法假设 percent 传入的是一个介于 0 和 1 之间的浮点数。在现实世界里这个假设可能过于激进（对大多数人来说，1000% 的涨薪大概是个 bug！）；对这个原型我们就放它一马，但在未来迭代这段代码时，我们可能想测试或至少记录这个假设。后续章节还会重

提这个想法，届时我们将探索函数装饰器（function decorators）和 Python 的 assert 语句——它们能在开发期间自动替我们做有效性检查。例如在第 39 章，我们会编写一个工具，允许我们用下面这样奇怪的咒语做校验：python @rangetest(percent=(0.0, 1.0)) # 用装饰器做校验def giveRaise(self, percent): self.pay = int(self.pay (1 + percent))

深度理解

·核心概念：封装（encapsulation）——把操作逻辑收进方法，全程序只留一份。方法是"挂"在类上的普通函数，self 是自动传入的实例（隐含主体）。

·底层实现：bob.lastName() 被解释器翻译成 Person.lastName(bob)——先在 bob 的继承链上找到 lastName 这个函数对象，再把它当作普通函数调用，并把 bob 作为第一个参数传入。这就是"方法 = 函数 + 自动传 self"的全部秘密。

·设计原因：消除冗余（redundancy）→ 可维护性（maintainability）。生活类比：把"顾客地址格式化"的规则写进一个函数，全公司（程序）共用；将来地址格式变了只改一处。

·实际问题：硬编码操作会"散落各处、改不胜改"；封装后还能让操作作用于任何实例——sue.lastName() 和 bob.lastName() 同一份代码、两个结果。

·初学者误区：① 忘了 return——方法变成返回 None；② 以为方法内可以直接用 name 而不写 self.name——

那会变成局部变量查找，找不到就报错或取错值；③ 以为 self 是关键字——它只是约定俗成的参数名，换成 this 也能运行。

代码分析

```
def lastName(self):                          # self
    return self.name.split()[-1]             #  self.name

def giveRaise(self, percent):
    self.pay = int(self.pay * (1 + percent))  #

#
print(bob.lastName(), sue.lastName())        #  bobsue  self
sue.giveRaise(.10)                           # 10%
```

·解释器如何处理：执行 bob.lastName() 时，字节码先加载 bob 对象，再执行 LOADMETHOD（Python 3.7+ 的优化指令：若名字来自类则直接绑定函数+实例，省一次属性查找），最后 CALL 时把 bob 放到 self 位置。int(self.pay (1 + percent))：先算浮点数乘积，再用 int() 截断小数——这就是输出从 110,000.00 变成 110000 的原因。

·设计思想：方法与调用方"解耦"——调用方不需要知道姓氏怎么提取、涨薪公式怎么写，只需知道接口 lastName() 与 giveRaise(percent)。这正是封装带来的接口稳定性。

Step 3: Operator Overloading（第三步：运算符重载）

英文原文

> At this point, we have a fairly full-featured class that generates and initializes instances, along with two ne w bits of behavior for processing instances in the for m of methods. So far, so good. As it stands, though, t esting is still a bit less convenient than it needs to be —to trace our objects, we have to manually fetch and print individual attributes (e.g., bob.name, sue.pay). It would be nice if displaying an instance all at once act ually gave us some useful information. Unfortunately, the default display format for an instance object isn't very good—it displays the object's class name and its address in memory (which is essentially useless in Pyt hon, except as a unique identifier). To see this, chang e the last line in the latest version of our script to pri nt(sue) so it displays the object as a whole. Here's wh at you'll get—as coded, the output says that sue is an object, but not much more: Bob Smith 0 Sue Jones 1 00000 Smith Jones <main.Person object at 0x104972 630>

中文翻译

此时，我们有一个功能相当完整的类：它能生成并初始化实例，还有两个以方法形式存在的新行为用来处理实例。到目前为止，一切顺利。不过就目前而言，测试仍然不够方便——要追踪我们的对象，必须手动取出并打印单个属性（比如bob.name、sue.pay）。如果能一次显示整个实例、而且

显示的内容确实有用，那就好了。遗憾的是，实例对象的默认显示格式并不好用——它只显示对象的类名和它在内存中的地址（在 Python 中这基本上毫无用处，充其量是个唯一标识符）。想看个明白，把最新版脚本的最后一行改成 print(sue)，让它整体显示这个对象。你会得到下面的结果——按现在的代码，输出只说明 sue 是一个对象，别的什么也没说： Bob Smith 0 Sue Jones 100000 Smith Jones <main.Person object at 0x104972630>

深度理解

·核心概念：默认的实例显示是 <main.Person object at 0x...>——类名 + 内存地址。这个地址在 Python 里（CPython 下）只是唯一标识，人类读者毫无意义。要获得有用的显示，需要运算符重载（operator overloading）。

·底层实现：print(sue) 最终调用 str(sue) → type(sue).str(sue)；若类没定义 str，则回退到 repr；再没有，就用 object 基类自带的默认 repr——它输出类名和 id 值（即内存地址的十进制或十六进制表示）。0x104972630 就是 CPython 中该对象在堆上的地址。

·设计原因：为什么默认不显示属性？因为解释器无法猜测你的对象"应该"长什么样——显示策略是领域相关的，必须由类的作者决定。

·实际问题：调试时需要快速看清对象状态；每次手打 print(bob.name, bob.pay) 既啰嗦又容易漏字段。

·初学者误区：以为打印对象就会"自动显示内容"——只有定义/继承 str 或 repr 才可能。

Providing Print Displays（提供打印显示）

英文原文

Fortunately, it's easy to do better by employing operator overloading—coding methods in a class that intercept and process built-in operations when run on the class's instances. Specifically, we can make use of what are probably the second most commonly used operator-overloading methods in Python, after init: the repr method we'll deploy here, and its str twin introduced in the preceding chapter. These methods are run automatically every time an instance is converted to its print string. Because that's what print normally does, the net transitive effect is that printing an object displays whatever is returned by the object's str or repr method, if the object either defines one itself or inherits one from a superclass. Double-underscored names are inherited just like any other. Technically, str is preferred by print and str, and repr is used both as a fallback for these, as well as by interactive echoes and nested appearances. Although the two can be used to implement different displays in different contexts (e.g., user- or developer-focused), coding repr alone suffices to give a single display in all cases. This allows clients to provide an alternative display with str, though this is a moot point for this demo. The init constructor method we've already coded is, strictly speak

ing, operator overloading too—it is run automatically at construction time to initialize a newly created instance. Constructors are so common, though, that they almost seem like a special case. More focused methods like repr allow us to tap into specific operations and provide specialized behavior when our objects are used in those contexts. Let's put this into code. Example 28-8 extends our class to give a custom display that lists attributes when our class's instances are displayed as a whole, instead of relying on the less useful default display.

Example 28-8: person8.py（添加 repr 重载方法用于显示）

```
class Person:
    def __init__(self, name, job=None, pay=0):
        self.name = name
        self.job = job
        self.pay = pay
    def lastName(self):
        return self.name.split()[-1]
    def giveRaise(self, percent):
        self.pay = int(self.pay * (1 + percent))
    def __repr__(self):                          #
        return f'[Person: {self.name} ${self.pay:,}]'   #
if __name__ == '__main__':
    bob = Person('Bob Smith')
    sue = Person('Sue Jones', job='dev', pay=100000)
    print(bob)
    print(sue)
    print(bob.lastName(), sue.lastName())
    sue.giveRaise(.10)
```

```
print(sue)
```

> Notice that we're doing f-string formatting to build the display string in repr here, with comma separators for pay; at the bottom, classes use built-in operations like these to get their work done. Again, everything you've already learned about both built-in objects and user-defined functions applies to class-based code. Classes largely just add an additional layer of structure that packages functions and data together and supports extensions. We've also changed our self-test code to print objects directly, instead of printing individual attributes. When run, the output is more uniform now; the "[...]" lines are returned by repr, run automatically by print operations:

> [Person: Bob Smith $0] [Person: Sue Jones $100,000] Smith Jones [Person: Sue Jones $110,000]

中文翻译

幸运的是，通过运用运算符重载（operator overloading）很容易做得更好——在类中编写一些方法，当内置操作作用于类的实例时，拦截并处理这些操作。具体来说，我们可以使用 Python 中仅次于 init 的、第二常用的运算符重载方法：本节要部署的 repr 方法，以及上一章介绍的它的孪生兄弟 str。每当实例被转换成它的打印字符串时，这些方法都会自动运行。因为 print 通常就是这么干的，所以净传递效果是：打印一个对象时，显示的就是该对象的 str 或

repr 方法返回的内容——前提是对象自己定义了其中一个，或者从某个超类继承了一个。双下划线名字和任何其他名字一样可以被继承。从技术上讲，print 和 str 优先使用 str，而 repr 既是它们的回退方案，也用于交互式回显和嵌套显示。虽然两者可以用来在不同的上下文里实现不同的显示（比如面向用户 vs 面向开发者），但只写 repr 就足以在所有情况下给出同一种显示。这样，客户还可以用 str 提供另一种显示——尽管对本演示来说这无关紧要。严格来说，我们已经写好的 init 构造方法也算运算符重载——它在构造阶段自动运行，用来初始化新建的实例。只不过构造函数太常见了，看起来几乎像个特例。像 repr 这样更有针对性的方法，允许我们介入特定操作，当对象用于这些场景时提供专门的行为。让我们把它写成代码。Example 28-8 扩展了我们的类，让类的实例整体显示时列出属性，而不是依赖没什么用的默认显示。注意我们在 repr 里用 f-string 格式化来构造显示字符串，pay 用了逗号千分位；说到底，类就是靠这样的内置操作来完成工作的。再说一次，你已经学过的关于内置对象和自定义函数的一切，都适用于基于类的代码。类基本上只是额外增加了一层结构，把函数和数据打包在一起并支持扩展。我们也把自测代码改成直接打印对象，而不是打印单个属性。运行时，输出现在更加统一；"[...]" 这些行就是 repr 返回的，由 print 操作自动调用： [Person: Bob Smith $0] [Person: Sue Jones $100,000] Smith Jones [Person: Sue Jones $110,000]

深度理解

·核心概念：运算符重载 = 为内置操作提供钩子方法。print、str、repr、+、[]、len() 等内置操作遇到类实例时，

会"打电话"给对应名字的双下划线方法。str / repr 是打印钩子，init 是构造钩子，add 是加法钩子。

·底层实现：print(sue) → 调用 str(sue) → 查找 type(sue).str，找不到回退到 type(sue).repr。查找发生在类上（而不是实例的 dict），所以继承的 repr 也能生效。交互式提示符敲 sue 回车，调用的则是 repr(sue) → repr。

·设计原因：为什么只写 repr 就够？因为 str()/print() 会先找 str，找不到就用 repr——一条 repr 通吃所有显示场景。而只写 str 的话，交互式回显仍会显示丑陋的默认形式。

·实际问题：调试友好。[Person: Sue Jones $110,000] 一行说清"谁、多少钱"，比内存地址有用得多。

·初学者误区：① 以为 repr 是"给程序员看"所以可以随便写——它确实要力求无歧义，能重建对象的信息量；② 忘了 return 而用 print 打印字符串——重载方法必须返回值；③ 以为 repr 只在 print 时调用——str()、repr()、f-string {}、列表显示等都会用到。

代码分析

```
def __repr__(self):                              #
    return f'[Person: {self.name} ${self.pay:,}]'   #


print(bob)        # print  __str__ __repr__
print(sue)        #  [Person: Sue Jones $100,000] :,
print(sue)        #  [Person: Sue Jones $110,000]
```

·解释器如何处理：print(bob) 的字节码调用 str(bob)，解释器沿 type(bob)（即 Person）的 MRO 查找 str——

没找到，回退找 repr，找到我们在类里定义的版本，把 bob 作为 self 传入。返回的 f-string 中 {self.pay:,} 是格式说明：逗号千分位。于是 100000 显示为 100,000。

·设计思想：把"对象如何自我呈现"封装进对象本身（repr），而不是每次打印时在外面拼字符串——又一次消灭冗余：显示逻辑只有一份，且对任何实例通用。

Step 4: Customizing Behavior by Subclassing（第四步：通过子类化定制行为）

英文原文

At this point, our class captures much of the OOP machinery in Python: it makes instances, provides behavior in methods, and even does a bit of operator overloading now to intercept print operations in repr. It effectively packages our data and logic together into a single, self-contained software component, making it easy to locate code and straightforward to change it in the future. By allowing us to encapsulate behavior, it also allows us to factor that code to avoid redundancy and its associated maintenance headaches. The only major OOP concept it does not yet capture is customization by inheritance. In some sense, we're already doing inheritance, because instances inherit methods from their classes. To demonstrate the real power of OOP, though, we need to define a superclass/subc

lass relationship that allows us to extend our softwar e and replace bits of inherited behavior. That's the m ain idea behind OOP, after all; by fostering a coding model based upon customization of work already do ne, it can dramatically cut development time when us ed well.

中文翻译

此时，我们的类已经捕获了 Python OOP 机制的大部分内容：它能创建实例，用方法提供行为，现在甚至还做了一点运算符重载，用 repr 拦截打印操作。它有效地把数据和逻辑打包成一个单一、自包含的软件组件（software compo nent），让代码容易定位、将来也容易修改。通过允许我们封装行为，它还允许我们分解代码，避免冗余及其相关的维护麻烦。它尚未捕获的唯一主要 OOP 概念是通过继承定制（customization by inheritance）。从某种意义上说，我们已经在做继承了，因为实例从它们的类继承方法。但要展示 OOP 真正的威力，我们需要定义一种超类/子类（sup erclass/subclass）关系，允许我们扩展软件、替换部分继承来的行为。毕竟这才是 OOP 背后的核心思想：通过培养"在已完成工作的基础上做定制"的编码模式，用得好时能大幅削减开发时间。

深度理解

·核心概念：本章前两步已经"隐式"地用到了继承（实例从类继承方法）；本节要建立显式的超类/子类关系，让子类替换（override）父类方法——这是 OOP 威力的真正来源。

·设计原因：OOP 的核心主张：不复制、不修改已有代码，而是"定制"已有代码。开发新需求时站在旧代码的肩膀上，而不是推翻重来。

·生活类比：手机厂商出一款"标准版"手机（Person），再出"Pro 版"（Manager）——机身、屏幕、系统几乎全继承，但相机（giveRaise）换了更强的算法。

·初学者误区：以为继承只用于"省事少打字"；其实继承的价值在于差异化管理——子类只写"不同之处"，相同之处自动来自父类。

Coding Subclasses（编写子类）

英文原文

As a next step, then, let's put OOP's methodology to use and customize our Person class by extending our software hierarchy. For the purpose of this tutorial, we'll define a subclass of Person called Manager that replaces the inherited giveRaise method with a more specialized version. Our new class begins as follows: python class Manager(Person): # Define a subclass of Person This code means that we're defining a new class named Manager, which inherits from and may add customizations to the superclass Person. In plain terms, a Manager is almost like a Person... (admittedly, a very long journey for a very small joke), but Manager has a custom way to give raises. For the sake of argument, let's assume that when a Manager gets a rai

se, it receives the passed-in percentage as usual, but also gets an extra bonus that defaults to 10%. For instance, if a Manager's raise is specified as 10%, it will really get 20%. (Any relation to Persons living or dead is, of course, strictly coincidental.) Our new method begins as follows; because this redefinition of giveRaise will be closer in the class tree to Manager instances than the original version in Person, it effectively replaces, and thereby customizes, the operation. Recall that according to the inheritance search rules, the lowest version of the name wins (we'll add this code to the file in a moment): python class Manager(Person): # Inherit Person attrs def giveRaise(self, percent, bonus=.10): # Redefine to customize

中文翻译

那么，作为下一步，让我们把 OOP 的方法论付诸实践，通过扩展软件层级来定制我们的 Person 类。为本次教程的目的，我们将定义一个 Person 的子类 Manager，用更专门的版本替换继承来的 giveRaise 方法。新类开头如下：python class Manager(Person): # 定义 Person 的子类这段代码的意思是：我们定义了一个名为 Manager 的新类，它继承自超类 Person，并可能对它做定制。通俗地说，Manager 几乎就是一个 Person……（承认吧，一个小玩笑走了很长的路），但 Manager 有自己独特的涨薪方式。为了讨论方便，让我们假设：当 Manager 涨薪时，它照常收到传入的百分比，但还会额外获得一个默认 10% 的奖金（bonus）。例如，如果给 Manager 指定的涨薪是 10%，它实

际上会得到 20%。（与任何在世或已故的 Person 的相似之处，当然纯属巧合。）我们的新方法开头如下；因为这个 giveRaise 的重定义在类树中比 Person 里的原版更接近 Manager 实例，它实际上替换了——从而定制了——这个操作。回忆一下继承查找规则：最低层的同名版本胜出（我们马上就把这段代码加进文件）：python class Manager(Person): # 继承 Person 的属性def giveRaise(self, percent, bonus=.10): # 重定义以定制

深度理解

·核心概念：class Manager(Person): 括号里的类就是超类。子类自动获得超类的全部属性和方法；若子类定义同名方法，则按继承查找规则"最低层胜出"——Manager 实例调 giveRaise 会命中 Manager 自己的版本，而不是 Person 的。

·底层实现：类对象创建后，Manager.bases 记录着 (Person,)；Manager 的 mro 列表是 [Manager, Person, object]。属性查找从实例 dict 开始，向上走 MRO 逐层找。这就是"继承搜索在对象树中查找属性"的具体执行。

·设计原因：默认参数 bonus=.10 让"经理多拿 10% 奖金"成为默认策略，同时保留调用方覆盖的可能——接口既有默认行为又可定制。

·初学者误区：以为 class Manager(Person) 是"复制"了 Person——不是，是引用：Person 的代码仍然只有一份，Manager 通过查找链共享它。这也意味着改 Person，Manager 自动跟着变。

Augmenting Methods: The Bad Way（增强方法：坏的做法）

英文原文

> Now, there are two ways we might code this Manage r customization: a good way and a bad way. Let's star t with the bad way since it might be a bit easier to un derstand. The bad way is to cut and paste the code o f giveRaise in Person and modify it for Manager, like this: python class Manager(Person): def giveRaise(self , percent, bonus=.10): self.pay = int(self.pay (1 + perc ent + bonus)) # Bad: cut and paste This would work a s advertised—when we later call the giveRaise metho d of a Manager instance, it would run this custom ver sion, which tacks on the extra bonus. So what's wron g with something that runs correctly? The problem h ere is a very general one: anytime you copy code wit h cut and paste, you essentially double your mainten ance effort in the future. Think about it: because we c opied the original version, if we ever have to change the way raises are given (and we probably will), we'll have to change the code in two places, not one. Alth ough this is a small and artificial example, it's also re presentative of a universal issue—anytime you're tem pted to program by copying code this way, you prob ably want to look for a better approach.

中文翻译

> 现在，我们有两种方式来编写这个 Manager 定制：一种好方式，一种坏方式。让我们从坏方式开始，因为它可能更容易理解。坏方式是把 Person 里 giveRaise 的代码剪切粘贴过来，再为 Manager 修改它，像这样：python class Manager(Person): def giveRaise(self, percent, bonus=.10): self.pay = int(self.pay (1 + percent + bonus)) # 坏做法：剪切粘贴这样会如广告所言地工作——之后调用 Manager 实例的 giveRaise 方法时，会运行这个定制版本，额外加上奖金。那么，能正确运行的东西有什么问题？这里的问题非常普遍：任何时候你用剪切粘贴复制代码，本质上都是把未来的维护工作量翻倍。想想看：因为我们复制了原版，将来若要改变涨薪的方式（而且我们很可能会改），就要改两处代码，而不是一处。虽然这是个很小的、人为构造的例子，但它代表了一个普遍问题——任何时候你动心想用这种方式复制代码来编程，你大概都应该寻找更好的方法。

深度理解

·核心概念：复制粘贴 = 代码冗余 = 维护负担翻倍。这是软件工程中最常见的"技术债"来源之一。

·底层实现：两处代码是两份独立的函数对象，互不关联——Python 没有任何机制让它们自动同步。

·设计原因：作者把它列为"坏方式"，因为：改涨薪规则时，Person 和 Manager 的版本各自独立演变，极易漏改一处，造成"普通员工和经理涨薪规则不一致"的隐蔽 bug。

·生活类比：公司两个部门各存一份《报销标准》复印件，财务新规出来后只改了行政部那份——研发部还在按旧规矩报销。单一事实来源（single source of truth）才是正道。

·初学者误区：以为"能跑 = 没问题"。能跑只是最低标准；可维护才是工程标准。

Augmenting Methods: The Good Way（增强方法：好的做法）

英文原文

> What we really want to do here is somehow augment the original giveRaise, instead of replacing it altogether. The good way to do that in Python is by calling to the original version directly, with augmented arguments, like this: python class Manager(Person): def giveRaise(self, percent, bonus=.10): Person.giveRaise(self, percent + bonus) # Good: augment original This code leverages the fact that a class's method can always be called either through an instance (the usual way, where Python sends the instance to the self argument automatically) or through the class (the less common scheme, where you must pass the instance manually). In more symbolic terms, a normal method call of this form: instance.method(args...) is automatically translated by Python into this equivalent form: class.method(instance, args...), where the class containing the

method to be run is determined by the inheritance search rule applied to the method's name. You can code either form in your script, but there is a slight asymmetry between the two—you must remember to pass along the instance manually if you call through the class directly. The method always needs a subject instance one way or another, and Python provides it automatically only for calls made through an instance. For calls through the class name, you need to send an instance to self yourself; and for code inside a method like giveRaise, self already is the subject of the call, and hence the instance to pass along. Calling through the class directly effectively subverts inheritance and kicks the call higher up the class tree to run a specific version. In our case, we can use this technique to invoke the default giveRaise in Person, even though it's been redefined at the Manager level. In fact, we must call through Person this way, because a self.giveRaise() inside Manager's giveRaise code would loop—since self already is a Manager, self.giveRaise() would resolve again to Manager's giveRaise, and so on, recursively, until available memory is exhausted. Call syntax aside, this "good" version may seem like a small difference in code, but it can make a huge difference for future code maintenance—because the giveRaise logic lives in just one place now (Person's method), we have only one version to change in the future as needs evolve. And really, this form captures our intent more

directly anyhow—we want to perform the standard gi veRaise operation, but simply tack on an extra bonus. Example 28-9 lists our entire module file with this ste p applied.

Example 28-9: person9.py（在子类中添加方法定制）

```
class Person:
    def __init__(self, name, job=None, pay=0):
        self.name = name
        self.job = job
        self.pay = pay
    def lastName(self):
        return self.name.split()[-1]
    def giveRaise(self, percent):
        self.pay = int(self.pay * (1 + percent))
    def __repr__(self):
        return f'[Person: {self.name} ${self.pay:,}]'

class Manager(Person):
    def giveRaise(self, percent, bonus=.10):        #
        Person.giveRaise(self, percent + bonus)     #  Person

if __name__ == '__main__':
    bob = Person('Bob Smith')
    sue = Person('Sue Jones', job='dev', pay=100000)
    print(bob)
    print(sue)
    print(bob.lastName(), sue.lastName())
    sue.giveRaise(.10)
    print(sue)
    pat = Manager('Pat Jones', 'mgr', 50000)        #  Manager…
    pat.giveRaise(.10)                              #
    print(pat.lastName())                           #
    print(pat)                                      #  __rep…
```

> To test our Manager subclass customization, we've also added self-test code that makes a Manager, calls its methods, and prints it. When we make a Manager, we pass in a name, and an optional job and pay as before—because Manager had no init constructor, it inherits that in Person. Here's the new version's output:

> [Person: Bob Smith $0] [Person: Sue Jones $100,000] Smith Jones [Person: Sue Jones $110,000] Jones [Person: Pat Jones $60,000]

> Everything looks good here: bob and sue are as before, and when pat the Manager is given a 10% raise, he or she really gets 20% (pay goes from $50K to $60K), because the customized giveRaise in Manager is run for this person only. Also notice how printing pat as a whole at the end of the test code displays the nice format defined in Person's repr: Manager objects get this, lastName, and the init constructor method's code "for free" from Person, by inheritance.

中文翻译

我们真正想做的是以某种方式增强（augment）原始的 giveRaise，而不是整个替换它。在 Python 中做这件事的好方式是直接调用原版，并传入增强过的参数，像这样：python class Manager(Person): def giveRaise(self, percent, bonus=.10): Person.giveRaise(self, percent + bonus) # 好做法：增强原版这段代码利用了一个事实：类的方法既可以通过实例调用（常见方式，Python 自动把实例传给 self 参数），也可以通过类调用（较不常见的方案，必须手

动传实例）。用更符号化的方式说，这种形式的普通方法调用：instance.method(args...)，会被 Python 自动翻译成等价形式：class.method(instance, args...)，其中包含要运行方法的类由对方法名应用继承查找规则来确定。你可以在脚本里编写任意一种形式，但两者之间有一点不对称——如果直接通过类调用，你必须记得手动把实例传进去。方法无论如何都需要一个主体实例，而 Python 只对通过实例发起的调用自动提供它。对于通过类名的调用，你需要自己把实例传给 self；而在像 giveRaise 这样的方法内部，self 已经就是这次调用的主体，因而也就是要传下去的实例。直接通过类调用，实际上是在"绕过"继承，把调用踢到类树更上层去运行一个特定版本。在我们的例子里，我们可以用这个技术去调用 Person 里默认的 giveRaise，即使它已经在 Manager 层被重定义了。事实上，我们必须这样通过 Person 调用，因为在 Manager 的 giveRaise 代码里写 self.giveRaise() 会造成死循环——因为 self 已经是一个 Manager，self.giveRaise() 会再次解析到 Manager 的 giveRaise，如此递归下去，直到可用内存耗尽。抛开调用语法不谈，这个"好"版本在代码上似乎只是小差别，但对未来的代码维护却影响巨大——因为 giveRaise 的逻辑现在只存在于一处（Person 的方法里），将来需求演变时我们只需改一个版本。而且说真的，这种形式本来就更直接地表达我们的意图——我们想做标准的 giveRaise 操作，只是额外附加一个奖金。Example 28-9 列出了应用这一步后的完整模块文件。为了测试我们的 Manager 子类定制，我们还添加了自测代码：创建一个 Manager，调用它的方法并打印它。创建 Manager 时，我们像之前一样传入名字、可选的 job 和 pay——因为 Manager 没有 init 构造方法，它继承了

Person 里的那个。以下是新版本的输出： [Person: Bob Smith $0] [Person: Sue Jones $100,000] Smith Jones [Person: Sue Jones $110,000] Jones [Person: Pat Jones $60,000] 一切看起来都很好：bob 和 sue 与之前一样；当 pat 这位 Manager 得到 10% 的涨薪时，他/她实际上得到了 20%（薪水从 5 万美元涨到 6 万美元），因为 Manager 里定制的 giveRaise 只对这个人生效。还要注意：测试代码末尾整体打印 pat 时，显示的是 Person 的 repr 里定义的漂亮格式——Manager 对象通过继承，"免费"从 Person 获得了 repr、lastName 和 init 构造方法的代码。

深度理解

·核心概念：增强（augment）而非替换。子类方法用 Person.giveRaise(self, percent + bonus) 显式调用父类版本，在它的基础上加功能。这揭示了方法调用的两种形式：instance.method(args) ≡ Class.method(instance, args)。

·底层实现：Person.giveRaise(self, ...) 是"通过类调用"——不触发继承查找，直接执行 Person 类里那个函数对象，并手动传 self。为什么不能写 self.giveRaise(...)？因为 self.giveRaise 会按继承查找命中 Manager 自己的版本——再调又是 Manager 版本——无限递归直到 RecursionError/内存耗尽。

·设计原因：单一事实来源：涨薪公式只在 Person 里存在一份；Manager 只是"加了一个奖金"的装饰者。以后改公式，Manager 自动跟上。

·实际问题：pat.giveRaise(.10) 中 .10 传给 Manager 版本，Manager 再以 percent + bonus（0.10 + 0.10 = 0.20）调 Person 版本——于是 50000 变 60000。输出验证：Manager 实例从继承链"免费"获得 lastName 和 repr（Manager 没定义，向上找到 Person 的）。

·初学者误区：① 在子类方法里写 self.giveRaise() 想"调父类版本"——那是递归自调；必须写类名.方法(self, ...)。② 以为 pat.lastName() 是"复制"来的——它是查找来的，代码只有一份。

代码分析

```
class Manager(Person):
    def giveRaise(self, percent, bonus=.10):   #  10%
        Person.giveRaise(self, percent + bonus)  # …


pat = Manager('Pat Jones', 'mgr', 50000)   #  __init__ Person …
pat.giveRaise(.10)                         #  Manager  20%
print(pat.lastName())                      # 'Jones'
print(pat)                                 #  __repr__[Person…
```

·解释器如何处理：pat.giveRaise(.10) 时，解释器沿 pat 的 MRO（[Manager, Person, object]）找 giveRaise，先命中 Manager 版本；执行时 Person.giveRaise(self, 0.20) 是普通函数调用，直接运行 Person 类的函数体，self 仍是 pat。结果 pat.pay = int(50000 1.20) = 60000。pat 自己没有 init，创建时继承查找命中 Person 的 init，所以 job='mgr'、pay=50000 正常初始化。

·设计思想：子类只表达"差异"（多一个 bonus），其余全部复用——这就是"定制已完成的工作"的编码模型。对比

坏方式，将来涨薪逻辑改动只落在一处。

The Super Alternative（super 替代方案）（侧栏）

英文原文

To extend inherited methods, the examples in this chapter simply call the original through the superclass name: Person.giveRaise(...). This is the explicit, traditional, and simplest scheme in Python, and the one used in most of this book. It also follows the same pattern we'll code to access class attributes: names attached to a class and shared by all instances, introduced ahead. Methods are just callable class attributes. Java programmers may especially be interested to know that Python also has a super built-in function that allows calling back to a superclass's methods more generically. In the custom giveRaise of Example 28-9's Manager class, both of the following would work the same way: python Person.giveRaise(self, percent + bonus) # Explicit, general use super().giveRaise(percent + bonus) # Implicit, special case The super built-in, however, relies on unusual semantics; works unevenly with Python's operator overloading; and does not always mesh well with multiple inheritance, where a single method call may not suffice. It's a special-case tool with a single role, which is redundant with general class-name references. In its defense, the super call has a

valid role too—cooperative same-named method dis patch in multiple inheritance trees—but this relies on the "MRO" class ordering of Chapter 31, which many find artificial; requires universal deployment to be us ed reliably; does not fully support method replaceme nt and varying argument lists; and to many observers seems an esoteric solution to a role that is rare in rea l Python code. Because of these downsides, this book prefers to call superclasses by explicit name instead o f super, recommends the same policy for newcomers, and defers presenting super until Chapter 32. You're of course free to draw your own conclusions there, b ut super is usually best had after you learn the simple r and more explicit ways of achieving the same goals in Python, especially if you're new to OOP. Topics like cooperative multiple-inheritance dispatch are a lot to digest—for beginners and experts alike. And to any J ava programmers in the audience: try resisting the te mptation to use Python's super until you've had a ch ance to study its subtle implications. This call is benig n in single-inheritance of the sort you're used to, but once you step up to Python's multiple inheritance, su per is not what you think it is, and more than you pro bably expect. The class it picks can vary per context, and may not even be a superclass at all!

中文翻译

为了扩展继承的方法，本章的例子只是通过超类名调用原版：Person.giveRaise(...)。这是 Python 中显式、传统、最简单的方案，也是本书大部分地方采用的方案。它还与我们将要编写的访问类属性（class attributes）的模式一致：挂在类上、被所有实例共享的名字，稍后介绍。方法只是可调用的类属性而已。Java 程序员可能尤其感兴趣：Python 也有一个 super 内置函数，可以用更通用的方式回调用超类的方法。在 Example 28-9 的 Manager 类的定制 giveRaise 中，下面两种写法效果相同：python Person.giveRaise(self, percent + bonus) # 显式、通用写法super().giveRaise(percent + bonus) # 隐式、特殊情况然而，super 内置函数依赖不寻常的语义；与 Python 的运算符重载配合得并不均衡；而且在多重继承下并不总能很好地工作——那里一次方法调用可能不够用。它是一个单一用途的特例工具，与通用的类名引用功能重复。为它辩护一句：super 调用也有正当的用武之地——多重继承树中同名方法的协作式分发（cooperative dispatch）——但这依赖第 31 章的 "MRO" 类顺序，很多人觉得它很人为；要可靠使用它必须全员部署；它不能完全支持方法替换和可变参数列表；而且在许多观察者看来，它似乎是对真实 Python 代码中罕见场景的一种玄学解法。由于这些缺点，本书更愿意用显式名字调用超类而不是 super，并向新手推荐同样的策略，把 super 的讲解推迟到第 32 章。你当然可以在那里自行得出结论，但对新手尤其如此：super 通常最好在学会了用更简单、更显式的方式达成同样目标之后再接触。像协作式多重继承分发这样的主题，无论对初学者还是专家都不好消化。还有给听众中的 Java 程序员的话：请尽量克制，在你研究过 super 的微妙含义之前不要使用 Python 的 super。在

> 你熟悉的单继承场景下这个调用是良性的，但一旦你进入 Python 的多重继承，super 就不是你想象的那个东西，而且比你预想的更复杂。它选择的类可能因上下文而变，甚至可能根本不是超类！

深度理解

·核心概念：super() 与显式类名调用。Person.giveRaise(self, ...)（显式、通用）vs super().giveRaise(percent + bonus)（隐式、特例）。本章明确主张：新手先用显式类名调用。

·底层实现：super() 是零参数形式，它根据 class 和 self 的类型计算出"当前类的后续 MRO"，从那里开始查找方法。单继承下结果与显式调用相同；多重继承下，super() 可能跳到"意想不到"的类——这就是作者警告"它可能根本不是超类"的原因。

·设计原因：作者的教学策略是先简单后复杂：显式类名调用零魔法、语义清晰；super 的协作式分发是高级场景的工具（第 31、32 章再展开）。

·实际问题：Java 程序员习惯 super.method()，但 Python 的 super() 语义更微妙（基于 MRO 而非父类），在运算符重载和多继承中行为不一致，所以本书劝大家"抵抗诱惑"。

·初学者误区：把 Python 的 super() 当作 Java 的 super 用；在多重继承里期待"父类"却被 MRO 里的"旁系"类惊到。

Polymorphism in Action（多态在行动）

英文原文

> To make this acquisition of inherited behavior even more striking, add the following code at the end of o ur file temporarily (this is file person9plus.py in the e xamples package, but doesn't warrant a full listing or caption here):

> python if name =='main': ... print('--All three--') for o bj in (bob, sue, pat): # Process objects generically obj .giveRaise(.10) # Run this object's giveRaise

> print(obj) # Run the common repr Here's the resultin g output, showing just its new parts (and the accumu lated effects of two raises per person):

> ...--All three-- [Person: Bob Smith $0] [Person: Sue Jo nes $121,000] [Person: Pat Jones $72,000]

In the added code, object is either a Person or a Ma nager, and Python runs the appropriate giveRaise aut omatically—our original version in Person for bob an d sue, and our customized version in Manager for pa t. Trace the method calls yourself to see how Python selects the right giveRaise method for each object. T his is just Python's notion of polymorphism, which w e met earlier in the book, at work again—what giveR aise does depends on what you do it to. Here, it's ma de all the more obvious when it selects from code we

've written ourselves in classes. The practical effect in this code is that sue gets another 10% but pat gets another 20% because giveRaise is dispatched based on the object's type. As we've seen, polymorphism is at the heart of Python's flexibility. Passing any of our three objects to a function that calls a giveRaise method, for example, would have the same effect: the appropriate version would be run automatically, depending on which type of object was passed. On the other hand, printing runs the same repr for all three objects, because it's coded just once in Person. Manager both specializes and applies the code we originally wrote in Person. Although this example is small, it's already leveraging OOP's talent for code customization and reuse; with classes, this almost seems automatic at times.

中文翻译

为了让这种"获得继承行为"的效果更震撼，让我们临时在文件末尾添加下面的代码（这是示例包里的 person9plus.py，但这里不值得给出完整列表或标题）：python if name =='main': ... print('--All three--') for obj in (bob, sue, pat): # 通用地处理对象obj.giveRaise(.10) # 运行这个对象的 giveRaise print(obj) # 运行共同的 repr 以下是结果输出，只显示它的新部分（以及每人两次涨薪的累积效果）：...--All three--[Person: Bob Smith $0] [Person: Sue Jones $121,000] [Person: Pat Jones $72,000] 在添加的代码里，object 要么是 Person，要么是 Manager，而 Py

thon 自动运行合适的 giveRaise——对 bob 和 sue 运行 Person 里的原版，对 pat 运行 Manager 里的定制版。请自己跟踪这些方法调用，看看 Python 是如何为每个对象选择正确的 giveRaise 方法的。这正是我们在书前面遇到过的 Python 的多态（polymorphism）概念在再次发挥作用——giveRaise 做什么取决于你把它用在了什么上。在这里，当它从我们自己写在类里的代码中选择时，这一点尤为明显。这段代码的实际效果是：sue 又涨了 10%，而 pat 又涨了 20%，因为 giveRaise 是根据对象的类型分发的。正如我们所见，多态是 Python 灵活性的核心。例如，把我们的三个对象中的任何一个传给一个调用 giveRaise 方法的函数，效果都一样：会根据传入的是哪种类型的对象，自动运行合适的版本。另一方面，打印对三个对象运行的 repr 是同一个——因为它只在 Person 里编写了一次。Manager 既特化又复用了我们最初写在 Person 里的代码。虽然这个例子很小，但它已经在利用 OOP 代码定制和复用的天赋了；有了类，这有时几乎像是自动发生的。

深度理解

·核心概念：多态（polymorphism）= "调用同一方法名，具体行为由对象的类型决定"。同一个 obj.giveRaise(.10) 循环体，对 Person 执行 Person 版本，对 Manager 执行 Manager 版本——分派（dispatch）发生在运行时。

·底层实现：循环每次迭代，obj.giveRaise 都是动态查找：解释器沿该对象类型的 MRO 找 giveRaise，找到哪个类的版本就用哪个。这是"鸭子类型"的类内体现——调用

方根本不知道（也不关心）对象是什么类，只要它有 giveRaise。

·设计原因：多态让"处理对象的通用代码"无需写 if/else 分支判断类型——扩展系统只需加新类，不用改旧代码（开闭原则）。这是 OOP 削减开发时间的关键机制。

·实际问题：sue 拿到 121000（110000×1.1），pat 拿到 72000（60000×1.2）——同一行 obj.giveRaise(.10) 生出两种结果。验证了多态确实是"类型驱动"的。另外注意：三个对象共享同一个 repr（只写一份），因为它没被重定义。

·初学者误区：以为多态需要"接口声明"或"父类指针"——Python 不需要；多态就是"按名查找，按类型分派"。另外注意 bob 的 0×1.1 还是 0——多态不会魔法般地修复数据问题。

代码分析

```
for obj in (bob, sue, pat):   #
    obj.giveRaise(.10)        #
    print(obj)                #  __repr__
```

·解释器如何处理：每次迭代，obj.giveRaise 都会沿 obj 的类型 MRO 重新查找：bob、sue 的类型是 Person，命中 Person 版；pat 的类型是 Manager，命中 Manager 版（多 10% 奖金）。这就是"动态分派"——查找发生在调用时，而非编译时。print(obj) 同理：三个对象的 str 都不存在，回退到 repr，沿各自 MRO 都找到 Person 的 repr，所以显示格式一致。

·设计思想：多态 = 同一接口、不同实现、自动选择。它是继承的"运行时受益面"：Manager 无需任何配合代码，就自动获得了"正确的涨薪"。

Inherit, Customize, and Extend（继承、定制与扩展）

英文原文

In fact, classes can be even more flexible than our example implies. In general, classes can inherit, customize, or extend existing code in superclasses. For example, although we're focused on customization here, we can also add unique methods to Manager that are not present in Person, if Managers require something different. The following abstract snippet illustrates. Here, giveRaise redefines a superclass's method to customize it, but someThingElse defines something new to extend: python class Person: def lastName(self): ... def giveRaise(self): ... def repr(self): ... class Manager(Person): # Inherit def giveRaise(self, ...): ... # Customize def someThingElse(self, ...): ... # Extend pat = Manager() pat.lastName() # Inherited verbatim pat.giveRaise() # Customized version pat.someThingElse() # Extension here print(pat) # Inherited overload method Extra methods like this code's someThingElse augment the existing software and are available on Manager objects only, not on Persons. For the purposes of this t

utorial, however, we'll limit our scope to customizing some of Person's behavior by redefining it, not addin g to it.

中文翻译

事实上，类比我们的例子所暗示的还要灵活。一般来说，类可以继承（inherit）、定制（customize）或扩展（exten d）超类中的现有代码。例如，虽然我们这里专注于定制，但如果 Manager 需要不同的东西，我们也可以给 Manag er 添加 Person 里没有的独有方法。下面这个抽象片段说明了这一点。这里，giveRaise 重定义了超类的方法来定制它，而 someThingElse 定义了新的东西来扩展：python class Person: def lastName(self): ... def giveRaise(self): ... def repr(self): ... class Manager(Person): # 继承def g iveRaise(self, ...): ... # 定制def someThingElse(self, ...): ... # 扩展pat = Manager() pat.lastName() # 原样继承pa t.giveRaise() # 定制的版本pat.someThingElse() # 在这里扩展print(pat) # 继承的重载方法像这段代码里的 some ThingElse 这样的额外方法增强了现有软件，并且只对 Ma nager 对象可用，对 Person 不可用。不过，就本教程而言，我们把自己限定在"通过重定义来定制 Person 的部分行为"，而不做添加。

深度理解

·核心概念：子类的三种能力：继承（inherit，原样复用）、定制（customize，同名重定义）、扩展（extend，新增独有方法）。三者经常组合出现。

·底层实现：pat.lastName() 在 Manager 的 MRO 里 Manager 层找不到 lastName，继续向上在 Person 层找到——"继承"就是这样发生的；pat.someThingElse() 在 Manager 层直接命中；Person 实例调用 someThingElse 则 AttributeError。

·设计原因：面向对象设计教条"继承是 is-a 关系"在这里落地：Manager is-a Person，所以能用 Person 的一切；Manager 又多出自己的职责（someThingElse）。

·实际问题：软件开发中 80% 的新需求是"旧行为 + 少量新行为"——继承/定制/扩展三件套正好覆盖。但注意：本章刻意只演示"定制"，把"扩展"留给你体会。

·初学者误区：以为子类必须"用上"父类的全部成员——完全不是；父类的东西你按需使用，子类的能力是超集（只增不减，除非刻意删除）。

OOP: The Big Idea（OOP：核心思想）

英文原文

As is, our code may be small, but it's fairly functional. And really, it already illustrates the main point behind OOP in general: in OOP, we program by customizing what has already been done, rather than copying or changing existing code. This isn't always an obvious win to newcomers at first glance, especially given the extra coding requirements of classes. But overall, the programming style implied by classes can cut develo

pment time radically compared to other approaches. For instance, in our example we could theoretically h ave implemented a custom giveRaise operation with out subclassing, but none of the other options yield c ode as optimal as ours:- Although we could have sim ply coded Manager from scratch as new, independen t code, we would have had to reimplement all the be haviors in Person that are the same for Managers.- Al though we could have simply changed the existing P erson class in place for the requirements of Manager' s giveRaise, doing so would break code that still nee ds the original Person behavior.- Although we could have simply copied the Person class in its entirety, re named the copy to Manager, and changed its giveRai se, doing so would introduce code redundancy that would double our work in the future—changes made to Person in the future would not be picked up auto matically, but would have to be manually propagated to Manager's code. As usual, the cut-and-paste appr oach may seem quick now, but it doubles your work i n the future. The customizable hierarchies we can bui ld with classes provide a much better solution for sof tware that will evolve over time. No other tools in Pyt hon support this development mode. Because we can tailor and extend our prior work by coding new subcl asses, we can leverage what we've already done, rath er than starting anew each time, breaking what alrea dy works, or introducing multiple redundant copies.

When done right, OOP is a powerful ally.

中文翻译

就目前而言，我们的代码可能很小，但相当实用。而且说真的，它已经阐明了 OOP 整体的核心思想：在 OOP 中，我们通过定制已完成的工作来编程，而不是复制或修改现有代码。对新来者来说，这乍一看不总是明显的胜利，尤其是考虑到类带来的额外编码要求。但总体而言，类所蕴含的编程风格可以与其他方法相比，从根本上削减开发时间。例如，在我们的例子里，理论上不通过子类化也能实现定制的 giveRaise 操作，但其他任何选项都产生不了我们这样最优的代码：- 虽然我们可以从零把 Manager 写成全新的、独立的代码，但我们得重新实现 Person 里那些对 Manager 也相同的行为。- 虽然我们可以为了 Manager 的 giveRaise 需求直接就地修改现有的 Person 类，但那样会破坏仍然需要原始 Person 行为的代码。- 虽然我们可以把 Person 类整个复制一份，把副本改名为 Manager 并修改它的 giveRaise，但那样会引入代码冗余，让将来的工作量翻倍——将来对 Person 的修改不会自动被 Manager 获得，而必须手动传播到 Manager 的代码里。和往常一样，剪切粘贴看起来现在很快，但将来要付出双倍代价。我们可以用类构建的可定制层级（customizable hierarchies），为"会随时间演化的软件"提供了好得多的解决方案。Python 中没有任何其他工具支持这种开发模式。因为我们可以通过编写新的子类来调整和扩展以前的工作，我们就能利用已经做好的东西，而不是每次从头再来、破坏已经能跑的东西、或者引入多份冗余副本。用得恰当时，OOP 是强大的盟友。

深度理解

·核心概念：OOP 的"大思想"——编程 = 定制已有代码，而非复制/修改/重写。作者对比了三个替代方案并逐一否决：从零重写（重复劳动）、就地修改父类（破坏他人）、复制改名（双份维护）。子类化是唯一零成本选项。

·底层实现：子类化不改动父类对象本身；父类代码在内存里只有一份，子类通过 MRO 引用它。因此"改动一处、全员生效"，且互不干扰。

·设计原因：软件唯一不变的是"会变"（evolution）。为演化而设计（design for change）是 OOP 的立身之本。作者强调"没有其他 Python 工具支持这种开发模式"——这是类不可替代的理由。

·生活类比：造车厂改一款新车型，不必重画底盘、也不必把老底盘改坏——"继承"老底盘设计，"定制"发动机，"扩展"自动驾驶套件。

·初学者误区：低估"改一处要手动同步到 N 处"的隐性成本。复制粘贴一时爽，维护火葬场——这是本书反复敲打的警钟。

Step 5: Customizing Constructors, Too（第五步：同样定制构造函数）

英文原文

Our code works as it is, but if you study the current v

ersion closely, you may be struck by something a bit odd—it seems pointless to have to provide a mgr job name for Manager objects when we create them: this is already implied by the class itself. It would be bett er if we could somehow fill in this value automatically when a Manager is made. The trick we need to impro ve on this turns out to be the same as the one we em ployed in the prior section: we want to customize the constructor logic for Managers in such a way as to pr ovide a job name automatically. In terms of code, we want to redefine an init method in Manager that pro vides the mgr string for us. And as in giveRaise custo mization, we also want to run the original init in Pers on by calling through the class name, so it still initiali zes our objects' state information attributes. The mo ds in Example 28-10 will do the job—we've coded th e new Manager constructor and changed the call that creates pat to not pass in the mgr job name.

Example 28-10: person10.py（在子类中定制构造函数）

```
class Person:
    def __init__(self, name, job=None, pay=0):
        self.name = name
        self.job = job
        self.pay = pay
    def lastName(self):
        return self.name.split()[-1]
    def giveRaise(self, percent):
        self.pay = int(self.pay * (1 + percent))
```

```
    def __repr__(self):
        return f'[Person: {self.name} ${self.pay:,}]'

class Manager(Person):
    def __init__(self, name, pay):              #
        Person.__init__(self, name, 'mgr', pay) #  job  '…
    def giveRaise(self, percent, bonus=.10):
        Person.giveRaise(self, percent + bonus)

if __name__ == '__main__':
    bob = Person('Bob Smith')
    sue = Person('Sue Jones', job='dev', pay=100000)
    print(bob)
    print(sue)
    print(bob.lastName(), sue.lastName())
    sue.giveRaise(.10)
    print(sue)
    pat = Manager('Pat Jones', 50000)           # job
    pat.giveRaise(.10)
    print(pat.lastName())
    print(pat)
```

Again, we're using the same technique to augment th e init constructor here that we used for giveRaise earl ier—running the superclass version by calling throug h the class name directly and passing the self instanc e along explicitly. Although the constructor has a stra nge name, the effect is identical. Because we need Pe rson's construction logic to run too (to initialize insta nce attributes), we really have to call it this way; othe rwise, instances would not have any attributes attach ed.

Calling superclass constructors from redefinitions thi s way turns out to be a very common coding pattern

in Python. By itself, Python uses inheritance to look f or and call only one init method at construction time —the lowest one in the class tree. If you need higher init methods to be run at construction time (and you usually do), you must call them manually, and usually through the superclass's name. The upside to this is t hat you can be explicit about which argument to pass up to the superclass's constructor and can choose to not call it at all: not calling the superclass constructor allows you to replace its logic altogether, rather than augmenting it.

The output of this file's self-test code is the same as before—we haven't changed what this example does, we've simply restructured it to get rid of some logical redundancy:

$ python3 person10.py [Person: Bob Smith $0] [Pers on: Sue Jones $100,000] Smith Jones [Person: Sue Jo nes $110,000] Jones [Person: Pat Jones $60,000]

Per the sidebar "The super Alternative", an implicit s uper().init(...) sans self would run the superclass const ructor too, and this is a common role for this built-in in single-inheritance class trees. For all the reasons n oted earlier, though, let's stick to the explicit and gen eral for now.

中文翻译

我们的代码现在能用了，但如果你仔细研究当前版本，可能

会注意到一件有点奇怪的事：创建 Manager 对象时还得手动提供 mgr 这个 job 名，似乎毫无必要——这本来就已经由类本身隐含了。如果创建 Manager 时能自动填上这个值，那就更好了。要改进这一点，我们需要的技巧与上一节用到的相同：以自动提供 job 名的方式定制 Manager 的构造逻辑。用代码说，就是重定义一个 Manager 的 init 方法，让它替我们填上 mgr 字符串。而且和 giveRaise 的定制一样，我们也要通过类名调用 Person 里原始的 init，让它继续初始化我们对象的状态信息属性。Example 28-10 的修改就能完成这个任务——我们编写了新的 Manager 构造函数，并修改了创建 pat 的调用，不再传入 mgr 这个 job 名。在这里，我们用的增强 init 构造函数的技巧和前面增强 giveRaise 的完全一样——直接通过类名调用超类版本，并显式把 self 实例传过去。虽然构造函数名字怪，但效果是一样的。因为我们也需要 Person 的构造逻辑运行（以初始化实例属性），真的必须这样调用；否则，实例上不会有任何属性被挂上。事实证明，这种从重定义里调用超类构造函数的方式是 Python 中非常常见的编码模式。单凭 Python 自己，在构造时只会通过继承查找并调用一个 init 方法——类树中最低的那个。如果你需要更高层的 init 方法也在构造时运行（而且你通常需要），就必须手动调用它们，通常通过超类的名字。这样做的优点是：你可以明确决定把哪些参数传给超类构造函数，甚至可以完全不调用它：不调用超类构造函数，意味着你可以整个替换它的逻辑，而不是增强它。这个文件的自测代码输出和之前一样——我们没有改变这个示例做什么，只是重构了它，去掉了一些逻辑冗余： $ python3 person10.py [Person: Bob Smith $0] [Person: Sue Jones $100,000] Smith Jones [Person: Sue J

ones $110,000] Jones [Person: Pat Jones $60,000] 按照侧栏 "The super Alternative" 的说法，隐式的 super().init(...)（不带 self）也会运行超类构造函数，而这正是这个内置函数在单继承类树里的常见用途。不过，出于前面提到的所有原因，眼下我们还是坚持显式、通用的写法。

深度理解

·核心概念：构造函数也可以被继承查找"遮住"——Python 构造时只调用 MRO 中最低的那个 init。子类重定义 init 后，父类初始化逻辑不会自动运行，必须手动调用（Person.init(self, ...)）。

·底层实现：pat = Manager('Pat Jones', 50000) 触发 Manager.init(pat,'Pat Jones', 50000)——查找在 Manager 层命中，Person 的 init 不会被自动执行。只有我们显式调用它，name/job/pay 三个属性才会被挂到 pat 的 dict 上。若不调用，pat 将"出生即裸奔"（无任何属性）。

·设计原因：为什么不让 Python 自动串联所有 init？因为显式优于隐式：让子类自己决定"是否调用父类构造、传什么参数、还是完全替换"——这给了最大灵活度。

·实际问题：消除逻辑冗余——'mgr' 这个字符串只出现在一处（Manager 自己的构造函数里），调用方 Manager('Pat Jones', 50000) 更简洁，且不可能写错 job 名。

·初学者误区：① 重定义 init 后忘记调用 Person.init，实例缺属性，访问时 AttributeError；② 以为子类构造时会"自动执行父类构造"——不会，必须手动；③ 误以为不写 init 的类"没有构造函数"——它会继承父类的。

代码分析

```
class Manager(Person):
    def __init__(self, name, pay):              #  name …
        Person.__init__(self, name, 'mgr', pay) #  job=…
    def giveRaise(self, percent, bonus=.10):
        Person.giveRaise(self, percent + bonus)

pat = Manager('Pat Jones', 50000)   # job  'mgr'
print(pat)                          # [Manager: job=mgr, name=Pat …
```

·解释器如何处理：Manager('Pat Jones', 50000) 沿 MRO 找到 Manager 的 init（它比 Person 的更"低"）；执行 Person.init(self, name,'mgr', pay) 时，三个赋值语句把'Pat Jones'、'mgr'、50000 写入 pat 的 dict。之后 pat.giveRaise(.10) → Manager 版本 → Person.giveRaise(pat, 0.20) → pay 变 60000。输出 [Manager: job=mgr, name=Pat Jones, pay=60000] 确认一切正确。

·设计思想：子类构造函数 = "默认参数注入 + 调用父类"，把领域知识（经理的 job 名必然是 mgr）封装在类内部，调用方无需关心。

OOP Is Simpler Than You May Think（OOP 比你想的更简单）

英文原文

> In this complete form, and despite their relatively sm
> all sizes, our classes capture nearly all the important
> concepts in Python's OOP machinery:- Instance creati

on—filling out instance attributes- Behavior methods—encapsulating logic in a class's methods- Operator overloading—providing behavior for built-in operations like printing- Customizing behavior—redefining methods in subclasses to specialize them- Customizing constructors—adding initialization logic to superclass steps Most of these concepts are based upon just three simple ideas: the inheritance search for attributes in object trees, the special self argument in methods, and operator overloading's automatic dispatch to methods. Along the way, we've also made our code easy to change in the future, by harnessing the class's propensity for factoring code to reduce redundancy. For example, we wrapped up logic in methods and called back to superclass methods from extensions to avoid having multiple copies of the same code. Most of these steps were a natural outgrowth of the structuring power of classes. By and large, that's all there is to OOP in Python. Classes certainly can become larger than this, and there are some more advanced class concepts, such as decorators and metaclasses, which we will explore in later chapters. In terms of the basics, though, our classes already do it all. In fact, if you've grasped the workings of the classes we've written, most OOP Python code should now be within your reach.

中文翻译

> 在这种完整形式下，尽管类的规模相对较小，它们已经捕获了 Python OOP 机制中几乎所有的重要概念：- 实例创建——填充实例属性- 行为方法——把逻辑封装进类的方法- 运算符重载——为打印等内置操作提供行为- 定制行为——在子类中重定义方法以专门化它们- 定制构造函数——在超类步骤的基础上添加初始化逻辑这些概念大多建立在仅仅三个简单想法之上：对象树中的继承属性查找、方法中特殊的 self 参数、以及运算符重载对方法的自动分派。一路走来，我们还通过利用类"分解代码以减少冗余"的天性，让代码将来容易修改。例如，我们把逻辑包进方法，并从扩展代码里回调超类方法，以避免同一代码的多份副本。这些步骤大多是类结构化能力自然生长的结果。总的来说，Python 的 OOP 就这么多。类当然可以变得比这更大，还有一些更高级的类概念，比如装饰器（decorators）和元类（metaclasses），我们将在后面的章节探索。但就基础而言，我们的类已经无所不包。事实上，如果你掌握了我们编写的这些类的运作方式，大多数 OOP Python 代码现在都应该在你的能力范围之内了。

深度理解

·核心概念：作者做了一个漂亮的归约：五大 OOP 概念（实例创建、行为方法、运算符重载、行为定制、构造函数定制）背后只有三条原理——继承查找、self 首参数、运算符重载自动分派。学 OOP 不必"背概念清单"，只需吃透这三个机制。

·底层实现：三个机制都是查找规则的不同应用：实例属性查 self.dict；方法查类型 MRO；+、print() 等内置操作查类上的特殊方法名。同一套"查表"逻辑贯穿始终。

·设计原因：作者一直在做减法——用小例子讲完大体系，降低心理负担。这也为第 29 章以后的语法深挖定了基调：你已经见过"全貌"，后面只是补充细节。

·实际问题：本章代码已经足够成为模板——任何"记录 + 处理"型应用（通讯录、库存、订单）都可以照此骨架改造。

·初学者误区：认为"OOP 就是类、继承、多态三大名词"——它们是效果而非机制；机制只有属性查找 + 特殊参数。想通这点，读任何 Python 类代码都不再发怵。

Other Ways to Combine Classes: Composites（组合类的其他方式：复合）

英文原文

> Having said that, it should be noted that although the basic mechanics of OOP are simple in Python, some of the art in larger programs lies in the way that classes are put together. We're focusing on inheritance in this tutorial because that's the mechanism the Python language provides, but programmers sometimes combine classes in other ways, too. For example, a common coding pattern involves nesting objects inside each other to build up composites. We'll explore this p

attern in more detail in Chapter 31, which is really m ore about design than about Python. As a quick exa mple, though, we could use this composition idea to code our Manager extension by embedding a Person, instead of inheriting from it. The alternative Manager in Example 28-11 does so in part by using the getattr operator-overloading method to intercept undefined attribute fetches, and the getattr built-in to delegate them to the embedded object: - The getattr method i sn't covered in full until Chapter 30, but its usage is si mple enough to employ here—it's automatically run for all fetches of undefined attributes (those not foun d in the instance by normal inheritance search). And yes, this is the same name used for the nonclass mod ule-attributes hook of Chapter 25. - The getattr call w as introduced in Chapter 25—it's the same as X.Y attr ibute fetch notation and thus performs inheritance, b ut the attribute name Y is evaluated to yield a string, not a name interpreted literally. More specifically her e, the constructor makes the embedded object with j ob name; giveRaise customizes by changing the argu ment passed along to the embedded object; and last Name triggers getattr to reroute to the embedded o bject. In effect, Manager becomes a controller layer t hat passes calls down to the embedded object, rather than up to superclass methods.

Example 28-11: person-composite.py（基于嵌入的 Manager 替代方案）

```
from person_10 import Person                # Example 28-10  …
class Manager:
    def __init__(self, name, pay):
        self.person = Person(name, 'mgr', pay)  #  Person
    def giveRaise(self, percent, bonus=.10):
        self.person.giveRaise(percent + bonus)  #
    def __getattr__(self, attr):
        return getattr(self.person, attr)       #
    def __repr__(self):
        return str(self.person)                 #
if __name__ == '__main__':
    pat = Manager('Pat Jones', 50000)           #  Person
    pat.giveRaise(.10)                          #  Manager.giveR…
    print(pat.lastName())                       #
    print(pat)                                  #  Manager.__rep…
```

The output of this version tests just its Manager class (the imported Example 28-10 tests its own code). As before, a 10% raise is munged to 20% by the custom giveRaise here, but other calls are handled by the em bedded Person, by either direct calls or generic attrib ute rerouting:

$ python3 person-composite.py Jones [Person: Pat J ones $60,000]

The more important point here is that this Manager alternative is representative of a general coding patte rn usually known as delegation—a composite-based structure that manages a wrapped object and propag ates method calls to it. This pattern works in our exa

mple, but it requires about twice as much code and is less well suited than inheritance to the kinds of direct customizations we meant to express. In fact, reasona ble Python programmers would almost certainly not code this example this way in practice. Manager isn't really a Person here, so we need extra code to manua lly dispatch method calls to the embedded object; op erator-overloading methods like repr must be redefin ed for reasons explained in the sidebar "Delegating B uilt-ins—or Not"; and adding new Manager behavior is less straightforward since state information is one l evel removed. Still, object embedding, and design pa tterns based upon it, can be a good fit when embedd ed objects require more limited interaction with the c ontainer than direct customization implies. A controll er layer, or proxy, like this alternative Manager, for ex ample, might come in handy if we want to adapt a cl ass to an expected interface it does not support, or tr ace or validate calls to another object's methods (ind eed, we will use a nearly identical coding pattern whe n we study class decorators later in the book). Moreo ver, a Department class prototype like that in Exampl e 28-12 could aggregate other objects in order to tre at them as a set.

Example 28-12: person-department.py（把嵌入对象聚合成复合）

```
from person_10 import Person, Manager    # Example 28-10
class Department:
    def __init__(self, *args):
        self.members = list(args)        #
    def addMember(self, person):
        self.members.append(person)
    def giveRaises(self, percent):       #
        for person in self.members:
            person.giveRaise(percent)
    def showAll(self):                   #
        for person in self.members:
            print(person)
if __name__ == '__main__':
    bob = Person('Bob Smith')
    sue = Person('Sue Jones', job='dev', pay=100000)
    pat = Manager('Pat Jones', 50000)
    development = Department(bob, sue)   #
    development.addMember(pat)
    development.giveRaises(.10)          #  giveRaise
    development.showAll()                #  __repr__
```

When run, the department's showAll method lists all of its contained objects after updating their state in t rue polymorphic fashion with giveRaises:

$ python3 person-department.py [Person: Bob Smith $0] [Person: Sue Jones $110,000] [Person: Pat Jones $ 60,000]

Interestingly, this code uses both inheritance and co mposition—Department is a composite that embeds and controls other objects to aggregate, but the emb edded Person and Manager objects themselves use i nheritance to customize. As another example, a GUI might similarly use inheritance to customize the beha

vior or appearance of labels and buttons, but also composition to build up larger packages of embedded widgets, such as input forms, calculators, and text editors. The class structure to use depends on the objects you are trying to model—in fact, the ability to model real-world entities this way is one of OOP's strengths. Design issues like composition are explored in Chapter 31, so we'll postpone further investigations for now. But again, in terms of the basic mechanics of OOP in Python, our Person and Manager classes already tell the entire story. Now that you've mastered the basics of OOP, though, developing general tools for applying it more easily in your scripts is often a natural next step—and the topic of the next section.

中文翻译

话虽如此，还应该指出：虽然 Python 中 OOP 的基本机制很简单，但大型程序中的一些艺术在于类如何组合。本教程聚焦继承（inheritance），因为那是 Python 语言提供的机制，但程序员有时也用其他方式组合类。例如，一种常见的编码模式是把对象嵌套进彼此，构建复合（composites）。我们将在第 31 章更详细地探讨这种模式，那一章实际上更关乎设计而非 Python。不过作为一个快速示例，我们可以用这种组合（composition）思想来编写 Manager 扩展——通过嵌入（embedding）一个 Person，而不是继承它。Example 28-11 中的替代 Manager 部分地借助了 getattr 运算符重载方法（拦截未定义属性的获取）和 getattr 内置函数（把属性委托给嵌入对象）来实现：- getattr

方法要到第 30 章才完整介绍，但它的用法简单到可以在这里使用——对所有未定义属性（按正常继承查找在实例中找不到的属性）的获取，它都会自动运行。是的，这与第 25 章非类模块属性钩子用的是同一个名字。- getattr 调用在第 25 章引入——它与 X.Y 属性获取记法相同，因此会执行继承查找，但属性名 Y 是被求值成字符串的，而不是字面解释的名字。具体到这里的代码：构造函数创建嵌入对象并指定 job 名；giveRaise 通过改变传给嵌入对象的参数来定制；lastName 触发 getattr 重新路由到嵌入对象。实际上，Manager 变成了一个控制器层（controller layer），把调用向下传给嵌入对象，而不是向上传给超类方法。这个版本的输出只测试它自己的 Manager 类（被导入的 Example 28-10 会测试它自己的代码）。和之前一样，定制的 giveRaise 把 10% 的涨薪变成 20%，但其他调用由嵌入的 Person 处理——要么直接调用，要么通过通用属性重路由： $ python3 person-composite.py Jones [Person: Pat Jones $60,000] 这里更重要的点是：这个 Manager 替代方案代表了一种通常称为委托（delegation）的通用编码模式——一种基于复合的结构，它管理一个被包裹的对象并向它传播方法调用。这种模式在我们的例子里能用，但它需要大约两倍的代码，而且对"我们想表达的这类直接定制"来说，不如继承合适。事实上，理性的 Python 程序员在实践中几乎肯定不会这样编写这个例子。这里的 Manager 并不是真正的 Person，所以我们需要额外代码手动把方法调用分派给嵌入对象；像 repr 这样的运算符重载方法必须重新定义（原因见侧栏 "Delegating Built-ins—or Not"）；而且添加新的 Manager 行为也不太直接，因为状态信息隔了一层。尽管如此，当嵌入对象与容器的交互比"直接定制"

更有限时，对象嵌入（object embedding）及基于它的设计模式可能非常合适。例如，像这个替代 Manager 这样的控制器层或代理（proxy），在我们想把一个类适配到它本不支持的预期接口、或者跟踪或校验对另一个对象方法的调用时，可能会派上用场（事实上，我们研究类装饰器（class decorators）时会用到几乎相同的编码模式）。此外，像 Example 28-12 那样的 Department 类原型可以把其他对象聚合（aggregate）起来，把它们当作一个集合来对待。运行时，部门的 showAll 方法会先用 giveRaises 以真正的多态方式更新所有内含对象的状态，然后列出它们： $ python3 person-department.py [Person: Bob Smith $0] [Person: Sue Jones $110,000] [Person: Pat Jones $60,000] 有意思的是，这段代码同时用了继承和组合——Department 是一个嵌入并控制其他对象以进行聚合的复合，但被嵌入的 Person 和 Manager 对象本身又用继承来定制。再举一个例子：GUI 可能同样用继承来定制标签和按钮的行为或外观，也用组合来构建更大的内嵌控件包，比如输入表单、计算器、文本编辑器。用哪种类结构取决于你要建模的对象——事实上，能这样为现实世界的实体建模正是 OOP 的优势之一。组合这类设计问题会在第 31 章探讨，所以我们先推迟进一步的调查。但再说一次，就 Python OOP 的基本机制而言，我们的 Person 和 Manager 类已经讲完了整个故事。不过，既然你已经掌握了 OOP 的基础，开发通用工具以便更轻松地在脚本中应用它，往往是顺理成章的下一步——这正是下一节的主题。

深度理解

·核心概念：组合（composition）/ 委托（delegation）

是继承之外的第二条"类组合"路线：不"is-a"，而是"has-a"——Manager 内部拥有一个 Person 对象，把调用转交给它。getattr 是"兜底属性钩子"：正常查找找不到属性时才被调用。

·底层实现：pat.lastName()：Manager 类里没有 lastName → 触发 getattr('lastName') → 调 getattr(self.person,'lastName') → 在嵌入的 Person 对象上查找并执行。pat.giveRaise(.10) 则直接命中 Manager 自己的版本，由它转发。注意 repr 必须显式重定义（见侧栏：内置操作不走 getattr）。

·设计原因：作者明确评价：本例中继承更好（代码减半、表达更直接）。但组合在"代理/适配/装饰"场景（接口适配、调用追踪、访问校验）中是不可替代的——这也是后面类装饰器的思想雏形。

·实际问题：Department 演示了聚合（aggregation）：把多个对象收进一个容器，统一调 giveRaises 批量涨薪——内部每个对象仍走自己的多态版本。GUI 编程（widget 树）就是组合的经典应用。

·初学者误区：① 以为组合"更高明"——它是另一种工具，各有适用场景（第 31 章详述）；② 以为 getattr 会拦截一切属性——它只拦"找不到的"；③ 以为 Manager "is-a" Person 就该永远用继承——设计判断要问"模型里 Manager 真的是 Person 吗"。

代码分析

```
class Manager:                                  #  Perso…
    def __init__(self, name, pay):
        self.person = Person(name, 'mgr', pay)  # self.person …
    def giveRaise(self, percent, bonus=.10):
        self.person.giveRaise(percent + bonus)  #
    def __getattr__(self, attr):
        return getattr(self.person, attr)       #
    def __repr__(self):
        return str(self.person)                 #  _…

class Department:
    def __init__(self, *args):
        self.members = list(args)               #
    def giveRaises(self, percent):              #
        for person in self.members:
            person.giveRaise(percent)           #
```

·解释器如何处理：pat.lastName() 时，Manager 实例的常规查找在类及 MRO 上找不到 lastName，解释器于是调用 Manager.getattr(pat,'lastName')，方法体里 getattr(self.person,'lastName') 再到 Person 实例上走一遍正常查找——找到 Person 的方法并执行。print(pat) 时，print 先找 str（没有），再找 repr——Manager 里定义了，直接命中，转发为 str(self.person)。

·设计思想：组合 = "包一层壳"：壳负责拦截/转发/定制，核心逻辑仍在嵌入对象里。继承把调用传"上"（super），组合把调用传"下"（嵌入对象）——两种方向两种哲学。

Delegating Built-ins—or Not（内置操作要不要委托）（侧栏）

英文原文

Notice how the delegation-based Manager class of Example 28-11 redefines the repr run by print, instead of allowing it to be caught by getattr like other attributes. In general, classes cannot intercept and delegate operator-overloading method attributes used by built-in operations without redefining them this way. Although we know that repr is the only such name used in our specific example, this is a broader issue for delegation-based classes. Recall that built-in operations like printing and addition implicitly invoke operator-overloading methods such as repr and add. Built-in operations like these, however, do not route their implicit attribute fetches through generic attribute managers: neither getattr (run for undefined attributes, and used here) nor getattribute (run for all attributes, and covered later) is invoked. Moreover, the implicit object superclass at the top of all class trees defines defaults that can preclude getattr too. Hence, Manager must redefine repr redundantly, in order to route printing to the embedded Person object. Comment out this class's repr method to see this live—the Manager instance then prints with a default instead of our custom display, because getattr is never run for printi

ng: $ python3 person-composite.py Jones <main.Manager object at 0x10a292c30> Technically, built-in operations begin their implicit search for method names at the class, skipping the instance entirely. By contrast, explicit by-name attribute fetches are always routed to the instance first. Classes also inherit a default repr from their automatic object superclass that would prevent getattr from running too, but the more absolute getattribute doesn't intercept the name either, and other methods not in object also won't be caught by getattr, like add for +. This was a mod in Python 3.X, but isn't a showstopper: delegation-based classes can still redefine operator-overloading methods to delegate them to wrapped objects, either manually or via tools or superclasses. It's also too advanced to explore further in this tutorial, but watch for it to crop up in Chapter 38, be solved in Chapter 39, and make a cameo appearance as a special case in the formal inheritance definition of Chapter 40.

中文翻译

注意 Example 28-11 基于委托的 Manager 类是如何重新定义 print 使用的 repr 的——而不是像其他属性那样让它被 getattr 捕获。一般来说，类不能拦截内置操作使用的运算符重载方法属性并委托它们，除非像这样重新定义。虽然我们知道 repr 是我们这个具体例子里唯一用到的这类名字，但这对基于委托的类来说是一个普遍问题。回想一下：打印、加法这样的内置操作会隐式调用 repr、add 这样的运

算符重载方法。然而，这样的内置操作不会把它们的隐式属性获取路由到通用属性管理器：getattr（为未定义属性运行，这里用到）和 getattribute（为所有属性运行，后面介绍）都不会被调用。此外，所有类树顶部的隐式 object 超类定义的默认方法，也可能阻止 getattr 生效。因此，Manager 必须冗余（redundantly）地重新定义 repr，才能把打印路由到嵌入的 Person 对象。把这个类的 repr 方法注释掉，就能现场看到这一点——Manager 实例随后会用默认显示而不是我们的自定义显示，因为打印时 getattr 永远不会运行： $ python3 person-composite.py Jones <main.Manager object at 0x10a292c30> 从技术上讲，内置操作对方法名的隐式查找是从类开始的，完全跳过实例。相比之下，显式的按名属性获取总是先路由到实例。类还会从自动的 object 超类继承一个默认 repr，它同样会阻止 getattr 运行；而更绝对的 getattribute 也不拦截这个名字；另外，object 里没有的其他方法（比如 + 用的 add）也不会被 getattr 捕获。这是 Python 3.X 的一个修改，但不是拦路虎：基于委托的类仍然可以通过重新定义运算符重载方法来委托给被包裹对象，无论是手动、借助工具还是通过超类。这个话题对本教程来说也过于高级，不便继续深入，但请注意它会在第 38 章再次出现、第 39 章得到解决，并在第 40 章形式化继承定义中作为特例客串登场。

深度理解

·核心概念：内置操作（print、+ 等）的隐式方法查找不走实例、也不走 getattr——它直接在类上找 str/repr/add 等特殊名字。这是"委托代理"类最大的坑。

·底层实现：print(pat) 的实现会直接查 type(pat)（即 Manager 类）的 MRO 找 repr——这条查找路径绕过了实例的 dict，也绕过了 getattr 钩子。Manager 类里没有，就去 object 超类拿默认的（显示类名+地址）。所以必须显式重定义 repr 才能拦截。

·设计原因：为什么这样设计？性能与一致性——内置操作在 CPython 内部走 C 级别的"类型槽位"（type slots）查找，天然与 Python 级属性钩子不同轨；同时避免 getattr 的通用逻辑干扰语言级操作的正确性。

·实际问题：写"包装类/代理类/装饰器类"时，凡是会用到内置操作的方法（repr、str、add、iter……）都要逐个显式转发——这是代理模式的固定成本。

·初学者误区：以为实现 getattr 就能"自动代理一切"——它代理不了内置操作；这也是很多代理类莫名其妙"打印出内存地址"的原因。

Step 6: Using Introspection Tools（第六步：使用内省工具）

英文原文

> Let's make one final tweak before we throw our objects onto a database. As they are, our classes are complete and demonstrate most of the basics of OOP in Python. They still have two remaining issues we probably should iron out, though, before we go live with the

m:- First, if you look at the display of the objects as t hey are right now, you'll notice that when you print p at the Manager, the display labels it as a Person. That 's not technically incorrect, since Manager is a kind of customized and specialized Person. Still, it would be more accurate to display an object with the most spe cific (that is, lowest) class possible: the one an object is made from.- Second, and perhaps more importantl y, the current display format shows only the attribute s we include in our repr, and that might not account f or future goals. For example, we can't yet verify that pat's job name has been set to mgr correctly by Man ager's constructor, because the repr we coded for Per son does not print this field. Worse, if we ever expan d or otherwise change the set of attributes assigned t o our objects in init, we'll have to remember to also u pdate repr for new names to be displayed, or it will b ecome out of sync over time. The last point means th at, yet again, we've made potential extra work for our selves in the future by introducing redundancy in our code. Because any disparity in repr will be reflected i n the program's output, this redundancy may be mor e obvious than the other forms we addressed earlier; still, avoiding extra work in the future is a generally g ood thing.

中文翻译

> 在我们把对象扔进数据库之前，做最后一个调整。就目前而言，我们的类已经完整，并演示了 Python OOP 的大部分基础。不过在正式投入使用之前，还有两个问题我们应该解决：- 第一，如果你看看现在对象的显示，会发现打印 pat 这位 Manager 时，显示把它标注成了 Person。从技术上说这不算错，因为 Manager 是一种定制、专门化的 Person。不过，用尽可能特定（即最低）的类来显示对象会更准确：也就是对象由哪个类制造出来的。- 第二，也许更重要：当前的显示格式只显示我们写进 repr 的属性，这可能无法照顾到未来的目标。例如，我们目前还无法验证 pat 的 job 名是否被 Manager 的构造函数正确设置成了 mgr，因为我们为 Person 写的 repr 不打印这个字段。更糟的是，如果我们将来在 init 里扩展或更改赋给对象的属性集合，就得记得同时更新 repr 以显示新名字，否则它迟早会失同步。最后一点意味着：我们又一次因为在自己的代码里引入冗余，而给未来制造了潜在的多余工作。因为 repr 的任何偏差都会反映在程序输出里，这种冗余可能比我们前面处理的其他形式更显眼；不过，避免未来的额外工作总的来说总是好事。

深度理解

·核心概念：现有 repr 有两个缺陷：① 硬编码类名'Person'——Manager 打印出来也叫 Person（不够精确）；② 硬编码属性列表——加新属性必须同步改 repr（重复劳动）。两个缺陷的共同根源：硬编码（hardcoding）。

·设计原因：这是"冗余"主题的第三次登场（硬编码操作 → 硬编码公式 → 硬编码显示）。作者想让学生形成直觉：任何"手工维护的同步关系"都是未来的坑。

·实际问题：加字段忘改显示，输出失真；类名写死，子类显示错误。这些在真实项目里会变成"数据显示不一致"的隐性 bug。

·生活类比：公司通讯录更新了"新部门"字段，但打印模板还是老的——新同事信息永远打不出来。模板应该"自动列出所有字段"。

·初学者误区：觉得"手写 repr 很麻烦，多写几个字段有什么关系"——问题不在当前字段，而在"以后每次改结构都要记着改它"。

Special Class Attributes（特殊的类属性）

英文原文

We can address both issues with Python's introspection tools—special attributes and functions that give us access to some of the internals of objects' implementations. These tools are somewhat advanced and generally used more by people writing tools for other programmers to use than by programmers developing applications. Even so, a basic knowledge of some of these tools is useful because they allow us to write code that processes classes in generic ways. In our code, for example, there are two hooks that can help us out, both of which were introduced near the end of the preceding chapter and used in earlier examples: - The built-in instance.class attribute provides a link from an instance to the class from which it was created.

Classes in turn have a name, just like modules, and a bases sequence that provides access to superclasses. We can use these here to print the name of the class from which an instance is made rather than one we've hardcoded. - The built-in object.dict attribute provides a dictionary with one key/value pair for every attribute attached to a namespace object (including modules, classes, and instances). Because it is a dictionary, we can fetch its keys list, index by key, iterate over its keys, and so on, to process all attributes generically. We can use this here to print every attribute in any instance, not just those we hardcode in custom displays, much as we did in Chapter 25's module tools. We met the first of these categories in the prior chapter, but here's a quick review at Python's interactive prompt with the latest versions of our person.py classes (Example 28-10). Notice how we load Person at the interactive prompt with a from statement here—class names live in and are imported from modules, exactly like function names and other variables:

>>> from person import Person >>> pat = Person('Pat Jones') >>> pat # Run pat's repr [Person: Pat Jones $0]

>>> pat.class # Show pat's class and its name <class' person.Person'> >>> pat.class.name'Person'

>>> list(pat.dict.keys()) # Attributes are dict keys ['name','job','pay']

>>> for key in pat.dict: print(key,'=>', pat.dict[key]) # Index manually: no inheritance name => Pat Jones

job => None pay => 0 >>> for key in pat.dict: # obj. attr, but attr is a string print(key,'=>', getattr(pat, key)) # Runs attr inheritance...same as prior output...

As noted briefly in the prior chapter, some attributes accessible from an instance might not be stored in th e dict dictionary if the instance's class defines a slots: an optional and relatively obscure feature of classes t hat stores attributes sequentially in the instance; may preclude an instance dict altogether; and which we w on't study in full until Chapter 32. Since slots really b elong to classes instead of instances, and since they are rarely used in any event, we can reasonably ignor e them here and focus on the normal dict. As we do, though, keep in mind that some programs may need to catch exceptions for a missing dict, or use built-in functions like hasattr, getattr, and dir if its users migh t deploy slots. As you'll learn in Chapter 32, the next section's code won't fail if used by a class with slots (i ts lack of them is enough to guarantee a dict) but slo ts—and other "virtual" attributes—won't be reported as instance data.

中文翻译

我们可以用 Python 的内省工具（introspection tools）解决这两个问题——这些特殊的属性和函数让我们能够访问对象实现的一些内部细节。这些工具有点高级，通常更多

是"为其他程序员编写工具的人"使用，而不是开发应用程序的程序员。即便如此，对其中一些工具的基础了解仍然有用，因为它们让我们能编写以通用方式处理类的代码。例如，在我们的代码里有两个钩子可以帮忙，它们都在前一章末尾介绍过，也在前面的例子里用过：- 内置的 instance.class 属性提供了从实例到"创建它的类"的链接。类反过来有 name（和模块一样），以及提供超类访问的 bases 序列。我们可以在这里用它们打印"创建实例的类"的名字，而不是我们硬编码的名字。- 内置的 object.dict 属性提供了一个字典，里面为命名空间对象（包括模块、类和实例）上挂的每个属性都保存一个键/值对。因为它是字典，我们可以取它的键列表、按键索引、遍历它的键等等，以通用方式处理所有属性。我们可以在这里用它打印任何实例的所有属性，而不只是我们在自定义显示里硬编码的那些——就像我们在第 25 章模块工具里做的那样。我们在前一章见过第一类工具，但这里用最新版 person.py 类（Example 28-10）在交互提示符下快速回顾一下。注意我们在这里用 from 语句在交互提示符下加载 Person——类名就活在被导入的模块里，和函数名及其他变量一模一样： >>> from person import Person >>> pat = Person('Pat Jones') >>> pat # 运行 pat 的 repr [Person: Pat Jones $0] >>> pat.class # 显示 pat 的类及其名字<class'person.Person'> >>> pat.class.name'Person'>>> list(pat.dict.keys()) # 属性就是字典的键['name','job','pay'] >>> for key in pat.dict: print(key,'=>', pat.dict[key]) # 手动索引：不做继承查找name => Pat Jones job => None pay => 0 >>> for key in pat.dict: # obj.attr 的等价写法，但 attr 是字符串 print(key,'=>', getattr(pat, key)) # 会执行属性继承...与

上一输出相同...如前一章简要提到的，如果实例的类定义了 slots，那么从实例可访问的一些属性可能不会存储在 dict 字典里：这是类的一个可选且相对冷门的特性，把属性按顺序存储在实例里；它可能完全取消实例的 dict；我们直到第 32 章才会完整研究它。由于 slots 实际上属于类而不是实例，而且无论如何都很少被使用，我们这里可以合理地忽略它们，专注正常的 dict。不过在这样做的时候请记住：如果用户可能使用 slots，有些程序可能需要为缺失的 dict 捕获异常，或者使用 hasattr、getattr、dir 等内置函数。你会在第 32 章学到：下一节的代码即使被带 slots 的类使用也不会失败（没有 slots 就足以保证有 dict），但 slots——以及其他"虚拟"属性——不会被报告为实例数据。

深度理解

·核心概念：内省（introspection）= 程序查看自身结构的能力。两个主力钩子：class（实例→类的链接）和 dict（实例的属性字典）。有了它们，"显示所有属性"就不再需要硬编码。

·底层实现：pat.dict 返回实例的真实属性字典 {'name':'Pat Jones','job': None,'pay': 0}——self.name = name 的最终归宿。pat.class 是指向 Person 类对象的引用；Person.name 是字符串'Person'。注意 getattr(pat, key) 与 pat.dict[key] 的区别：前者走完整属性查找（含继承），后者只查实例字典。

·设计原因：作者点出内省工具的定位：给"造工具的人"用的——通用工具（如通用显示器、ORM、调试器）必须能处理"未知结构"的对象，硬编码会杀死通用性。

·实际问题：class 解决"显示真实类名"；dict 解决"显示所有字段"。二者组合，就是下一节 AttrDisplay 通用显示工具的基础。

·初学者误区：① 以为 dict 只对实例有效——模块、类、函数都有 dict；② 以为 dir(pat) 与 list(pat.dict) 等价——dir 包含继承的名字，dict 只有自己的；③ 忘了 slots 类没有实例 dict（第 32 章详解）。

代码分析

```
pat.__class__                    #
pat.__class__.__name__           # 'Person'
list(pat.__dict__.keys())        # ['name', 'job', 'pay']
for key in pat.__dict__:         #
    print(key, '=>', pat.__dict__[key])   #
    print(key, '=>', getattr(pat, key))   #
```

·解释器如何处理：pat.class 是一个直接存储在实例上的内置链接（C 层字段），返回类对象；pat.class.name 再从类对象的属性里取字符串。getattr(pat, key) 是内置函数调用，内部执行与 pat.key 相同的属性查找算法（实例 → 类 → MRO），区别只是名字来自运行时字符串——这正是"通用代码"的前提。

·设计思想："数据驱动显示"替代"手工编码显示"：显示什么由对象自己的结构决定，而不是由作者的记忆决定。这就是消除 repr 冗余的钥匙。

A Generic Display Tool（通用显示工具）

英文原文

We can put these interfaces to work in a superclass t
hat displays accurate class names and formats all attr
ibutes of an instance of any class. This superclass is c
oded in Example 28-13—it's a new, independent mo
dule named classtools.py. Because its repr display ov
erload uses generic introspection tools, it will work o
n any instance, regardless of the instance's attributes
set. And because this is a class, it automatically beco
mes a general formatting tool: thanks to inheritance,
it can be mixed into any class that wishes to use its di
splay format. As an added bonus, if we ever want to c
hange how instances are displayed we need only cha
nge this class—every class that inherits its repr will a
utomatically pick up the new format when next used
by a program. Notice the docstrings here—because t
his is a general-purpose tool, we want to add some f
unctional documentation for potential users to read.
As we saw in Chapter 15, docstrings can be placed at
the top of simple functions and modules, and also at
the start of classes and any of their methods; the hel
p function and the PyDoc tool extract and display the
se automatically. We'll revisit docstrings for classes in
Chapter 29. When run directly, this module's self-test
makes two instances and prints them; the repr define
d here shows the instance's class, and all its attribute
s' names and values, in sorted attribute name order:

$ python3 classtools.py [TopTest: attr1=0, attr2=1] [SubTest: attr1=2, attr2=3] Another design note here: because this class uses repr instead of str its displays are used in all contexts, but its clients also won't have the option of providing an alternative low-level display—they can still add a str, but this applies to print and str only. In a more general tool, using str instead limits a display's scope, but leaves clients the option of adding a repr for a secondary display at interactive prompts and nested appearances. We'll follow this alternative policy when we code expanded versions of this class in Chapter 31; for this demo, we'll stick with the all-inclusive repr.

Example 28-13: classtools.py（新文件）

```
"Assorted class utilities and tools"   #

class AttrDisplay:
    """


     name=value
    mixin
    """
    def gatherAttrs(self):
        attrs = []
        for key in sorted(self.__dict__):      #
            attrs.append(f'{key}={getattr(self, key)}')
        return ', '.join(attrs)                #  'name=value, j…
    def __repr__(self):
        return f'[{self.__class__.__name__}: {self.gatherAttrs()}]'

if __name__ == '__main__':
```

```
class TopTest(AttrDisplay):
    count = 0                                #
    def __init__(self):
        self.attr1 = TopTest.count           #
        self.attr2 = TopTest.count+1
        TopTest.count += 2                   #
class SubTest(TopTest):
    pass                                     #

X, Y = TopTest(), SubTest()                  #
print(X)                                     #
print(Y)                                     #
```

中文翻译

我们可以把这些接口用到一个超类中：它显示准确的类名，并格式化任何类的实例的所有属性。这个超类写在 Example 28-13 里——它是一个新的、独立的模块，名叫 classtools.py。因为它的 repr 显示重载用的是通用内省工具，所以它对任何实例都有效，无论该实例的属性集合是什么。又因为它是一个类，它自动变成通用的格式化工具：多亏了继承，它可以被混入（mixed into）任何希望使用这种显示格式的类。额外的好处是：如果将来想改变实例的显示方式，只需改这一个类——所有继承它的 repr 的类，下次被程序使用时都会自动采用新格式。注意这里的 docstring（文档字符串）——因为这是一个通用工具，我们要为潜在用户添加一些功能性文档供阅读。正如我们在第 15 章看到的，docstring 可以放在简单函数和模块的顶部，也可以放在类的开头以及类的任何方法的开头；help 函数和 PyDoc 工具会自动提取并显示它们。我们将在第 29 章再次讨论类的 docstring。直接运行时，这个模块的自测会创建两个实例并打印它们；这里定义的 repr 会显示实例的类，以及它所

有属性的名字和值，按属性名的排序顺序： $ python3 classtools.py [TopTest: attr1=0, attr2=1] [SubTest: attr1=2, attr2=3] 这里还有一个设计说明：因为这个类用 repr 而不是 str，它的显示在所有上下文里都有效，但它的客户也无法选择提供另一种底层显示——他们仍然可以添加 str，但那只对 print 和 str 生效。在更通用的工具里，改用 str 会限制显示的作用范围，但给客户留下添加 repr 以在交互提示符和嵌套显示中提供次要显示的选择。我们在第 31 章编写这个类的扩展版本时会采用这种替代策略；本演示中，我们坚持使用包罗万象的 repr。

深度理解

·核心概念：AttrDisplay = 通用显示工具（mixin 类）。它把"显示对象"这个职责从具体业务类中抽出来，做成一个谁都能继承的通用组件——这是"工具类"与"业务类"分离的典范。

·底层实现：self.dict 拿到实例属性字典，sorted() 排序保证输出稳定，getattr(self, key) 取值，f-string 拼出 key=value；self.class.name 动态取类名——Manager 显示为 Manager，Person 显示为 Person，永远准确。自测里的 count 是类属性（挂在类上），不在实例 dict 里，所以不会显示。

·设计原因：一次编码、到处继承（DRY 原则的最高形态）：将来改显示格式只动这一处；用 repr 而非 str 则让显示在所有场景生效（print、str、交互回显、嵌套显示）。

·实际问题：SubTest 一行 pass 就继承了完整显示能力——这是 mixin 模式的核心价值；count 计数演示"类属性共享、实例属性私有"。

·初学者误区：① 以为 sorted(self.dict) 排序的是"属性对象"——它排序的是字符串键名；② 以为混入的类方法会"复制"进子类——不会，还是靠查找；③ 忘了 getattr(self, key) 与 self.dict[key] 的差别（前者含继承，后者不含）——对普通实例属性二者相同，但语义不同。

代码分析

```
class AttrDisplay:
    def gatherAttrs(self):
        attrs = []
        for key in sorted(self.__dict__):     #
            attrs.append(f'{key}={getattr(self, key)}')  #  getat…
        return ', '.join(attrs)               #  'attr1=0, attr…
    def __repr__(self):
        return f'[{self.__class__.__name__}: {self.gatherAttrs()}]…

X, Y = TopTest(), SubTest()    # X: attr1=0, attr2=1Y: attr1=2, a…
print(X)                       # [TopTest: attr1=0, attr2=1]
print(Y)                       # [SubTest: attr1=2, attr2=3] — …
```

·解释器如何处理：print(X) → 找 str（无）→ 找 repr，在 AttrDisplay 里命中 → 以 X 为 self 执行：self.class.name 求值为'TopTest'；gatherAttrs() 遍历 X 的 dict（只有 attr1、attr2——count 在类上不在实例里），按排序生成 attr1=0, attr2=1。SubTest 没有自己的 init，创建时沿 MRO 找到 TopTest 的 init，count 从 0 递增到 2，所以 Y 得到 attr1=2、attr2=3。

·设计思想：通用性来自"读取对象自身结构"而非"记住对象结构"——无论实例有几个属性、叫什么名，都能正确显示。显示逻辑与业务逻辑解耦，可复用、可独立演化。

Instance Versus Class Attributes（实例属性与类属性）

英文原文

If you study the classtools module's self-test code long enough, you'll notice that its class displays only instance attributes, attached to the self object at the bottom of the inheritance tree; that's what self's dict contains. As an intended consequence, we don't see attributes inherited by the instance from classes above it in the tree. For example, the attribute TopTest.count used as an instance counter in this file's self-test code is a class attribute that lives only on the class: it's assigned in the class block like methods, and referenced in methods by explicit class name—much like explicit calls to customized class methods. It's also inherited by instances, but inherited class attributes are attached to the class only, not copied down to instances. There's more on such nonmethod class attributes in the next chapter; the main point here is that count won't be displayed by AttrDisplay. If you ever do wish to include inherited attributes too, you can climb the class link to the instance's class, use the dict there t

o fetch class attributes, and then iterate through the class's bases attribute to climb to even higher superclasses, repeating as necessary. If you're a fan of simple code, running a built-in dir call on the instance instead of using dict and climbing would have much the same effect, since dir results include inherited names in sorted results lists (along with built-in X methods that are easy to filter out as usual): $ python3 >>> from person10 import Person >>> pat = Person('Pat Jones') >>> list(pat.dict) ['name','job','pay'] >>> [a for a in dir(pat) if not a.startswith('')] ['giveRaise','job','lastName','name','pay'] In the interest of space, we'll leave optional display of inherited class attributes with either tree climbs or dir as suggested experiments for now. For more hints on this front, though, watch for the classtree.py inheritance-tree climber we will write in Chapter 29, and the lister.py attribute listers and climbers we'll code in Chapter 31.

中文翻译

如果你仔细研究 classtools 模块的自测代码，你会注意到它的类只显示实例属性——挂在继承树底部的 self 对象上的那些；self 的 dict 装的就是它们。作为有意的后果，实例从树上方的类继承来的属性，我们看不到。例如，这个文件自测代码里用作实例计数器的属性 TopTest.count 就是一个类属性（class attribute），只存在于类上：它像方法一样在类块里赋值，在方法里通过显式的类名引用——很像对定制类方法的显式调用。实例也会继承它，但被继承的

类属性只挂在类上，不会被复制到实例。下一章还有更多关于这种非方法类属性的内容；这里的主要观点是：count 不会被 AttrDisplay 显示出来。如果你确实想把继承来的属性也包含进来，可以沿 class 链接爬到实例的类，用那里的 dict 取类属性，然后遍历类的 bases 属性爬到更高的超类，必要时重复这个过程。如果你喜欢简单的代码，在实例上调用内置 dir 而不是用 dict 加爬树，效果也差不多——因为 dir 的结果按排序列表包含继承来的名字（还有一些内置 X 方法，和往常一样很容易过滤掉）： $ python3 >>> from person10 import Person >>> pat = Person('Pat Jones') >>> list(pat.dict) ['name','job','pay'] >>> [a for a in dir(pat) if not a.startswith('')] ['giveRaise','job','lastName','name','pay'] 为了节省篇幅，把"可选地显示继承的类属性"（无论用爬树还是 dir）留作建议的实验吧。不过关于这方面还有更多提示：注意我们将在第 29 章编写的 classtree.py 继承树爬行者，以及我们将在第 31 章编写的 lister.py 属性列示与爬行工具。

深度理解

·核心概念：实例属性 vs 类属性——实例属性在实例 dict 里，每实例一份；类属性在类对象的 dict 里，所有实例共享一份。AttrDisplay 只显示前者，是有意为之。

·底层实现：TopTest.count 在类块里赋值，所以它存在于 TopTest.dict 的 {'count': ...} 中；实例读取 X.count 时，实例字典没有，就沿 MRO 去类上找——"继承"就是这么实现的，但不会复制到实例字典。dir(pat) 返回排序后的、包含继承名的列表；pat.dict 只有自己的三个键。

·设计原因：共享状态放类属性（如计数器、常量），独立状态放实例属性（name、pay）——这是"数据该放哪"的经典判断：该对象独有还是全体共享。

·实际问题：如果想把类属性也显示出来，可以沿 class → dict → bases 爬树，或直接用 dir——第 29、31 章会给出正式工具（classtree.py、lister.py）。

·初学者误区：以为 X.count = 5 会"改掉类的 count"——不，它只是给 X 这个实例新建了一个同名属性，遮住了类属性；这是新手最容易踩的类属性陷阱。

Name Considerations in Tool Classes（工具类中的命名考量）

英文原文

> One last subtlety here: because our AttrDisplay class in the classtools module is a general tool designed to be mixed into other arbitrary classes, we have to be aware of the potential for unintended name collisions with client classes. As is, the code assumes that client subclasses may want to use both its repr and gatherAttrs, but the latter of these may be more than a subclass expects—if a subclass innocently defines a gatherAttrs name of its own, it will likely break our class, because the lower version in the subclass will be used instead of ours. To see this for yourself, add a gatherAttrs to TopTest in the file's self-test code; unless the new method is identical, or intentionally customizes t

he original, our tool class will no longer work as planned—self.gatherAttrs within AttrDisplay searches anew from the TopTest instance: python class TopTest(AttrDisplay):...def gatherAttrs(self): # Replaces method in AttrDisplay! return'Oops' This isn't necessarily bad—sometimes we want other methods to be available to subclasses, either for direct calls or for customization this way. If we really meant to provide a repr only, though, this is less than ideal. To minimize the chances of name collisions like this, Python programmers often prefix methods not meant for external use with a single underscore: gatherAttrs in our case. This isn't foolproof (what if another class defines its own "private" gatherAttrs, too?), but it's usually sufficient, and it's a common Python naming convention for methods internal to a class. A better solution would be to use two underscores at the front of the method name only: gatherAttrs for us. Python automatically expands such names to include the enclosing class's name, which makes them truly unique when looked up by the inheritance search. This is a feature usually called pseudoprivate class attributes, which we'll expand on in Chapter 31 and deploy in an expanded version of this class there. For now, we'll make both our methods available.

中文翻译

这里还有最后一个微妙之处：因为 classtools 模块里的 AttrDisplay 类是一个设计用来混入其他任意类的通用工具，我们必须意识到与客户类发生意外名字冲突（name collisions）的可能性。就目前而言，代码假设客户子类可能想同时使用它的 repr 和 gatherAttrs，但后者可能超出子类的预期——如果某个子类无意中定义了自己的 gatherAttrs 名字，它很可能会破坏我们的类，因为查找时会使用子类里更低的那个版本，而不是我们的。想亲眼看看，就在文件自测代码里给 TopTest 加一个 gatherAttrs；除非新方法与原来的相同，或者有意定制原版，否则我们的工具类就不会按计划工作了——AttrDisplay 内部的 self.gatherAttrs 会从 TopTest 实例重新开始查找：python class TopTest(AttrDisplay):...def gatherAttrs(self): # 替换了 AttrDisplay 里的方法！return'Oops'这不一定坏——有时候我们就是想让其他方法对子类可用，无论是直接调用还是这样定制。不过，如果我们真的只想提供一个 repr，那就不太理想了。为了尽量减少这种名字冲突的可能，Python 程序员经常给不打算对外使用的方法加上单下划线前缀：在我们这里就是 gatherAttrs。这并不万无一失（如果别的类也定义了自己的"私有" gatherAttrs 呢？），但通常足够了，而且是 Python 中类内部方法的常见命名约定。更好的解决方案是只在方法名开头用双下划线：对我们来说就是 gatherAttrs。Python 会自动把这样的名字扩展为包含所在类的名字，使它们在继承查找中真正唯一。这个特性通常称为伪私有类属性（pseudoprivate class attributes），我们将在第 31 章详细展开，并在那里部署这个类的扩展版本。眼下，我们让两个方法都可使用。

深度理解

·核心概念：混入工具类的"命名卫生"。AttrDisplay 内部调用 self.gatherAttrs()——这个调用是动态查找的，如果子类定义了同名方法，就会"截胡"（被子类的版本覆盖）。这就是 mixin 设计的固有风险：内部方法名必须"足够独特"。

·底层实现：单下划线 gatherAttrs 只是约定（对解释器无任何特殊意义，仅在 from-import 时被过滤）；双下划线 gatherAttrs 则有真正的机制：解释器在类定义时把 gatherAttrs 改写（name mangling）成 AttrDisplaygatherAttrs——名字里烙上类名，别的类很难撞车。

·设计原因：工具类要为"任何客户的任何类"服务，因此要最小化对客户命名空间的假设；这是库作者的责任心。

·实际问题：如果你写了一个"通用工具类"给团队用，被同事的子类方法不小心覆盖导致诡异 bug——命名规范和伪私有就是防这个的。

·初学者误区：以为双下划线是"真正的私有"（像 Java 的 private）——不是，它只是改名；通过 AttrDisplaygatherAttrs 仍然可以访问。它是"防误撞"而非"防访问"。

Our Classes' Final Form（我们类的最终形态）

英文原文

Now, to use the preceding section's generic display tool in our classes, all we need to do is import it from

its module, mix it in by inheritance in our top-level class, and get rid of the more specific repr we coded before. The new display overload method will be inherited by instances of Person, as well as Manager; Manager gets repr from Person, which now obtains it from the AttrDisplay coded in another module. Example 28-14 codes the final version of our person.py file with these changes applied, and imports and uses the separate Example 28-13.

Example 28-14: person14.py（最终版）

```
"""



"""
from classtools import AttrDisplay        #

class Person(AttrDisplay):                #  repr
    """

    """
    def __init__(self, name, job=None, pay=0):
        self.name = name
        self.job = job
        self.pay = pay
    def lastName(self):                   #
        return self.name.split()[-1]
    def giveRaise(self, percent):         # percent  0..1
        self.pay = int(self.pay * (1 + percent))

class Manager(Person):
    """
     Person
    """
```

```
    def __init__(self, name, pay):
        Person.__init__(self, name, 'mgr', pay)   # job
    def giveRaise(self, percent, bonus=.10):
        Person.giveRaise(self, percent + bonus)

if __name__ == '__main__':
    bob = Person('Bob Smith')
    sue = Person('Sue Jones', job='dev', pay=100000)
    print(bob)
    print(sue)
    print(bob.lastName(), sue.lastName())
    sue.giveRaise(.10)
    print(sue)
    pat = Manager('Pat Jones', 50000)
    pat.giveRaise(.10)
    print(pat.lastName())
    print(pat)
```

> As this is the final revision, we've added a few comm ents here to document our work—docstrings for func tional descriptions and # for smaller notes, per best-practice conventions, as well as blank lines between methods for readability—a generally good style choi ce when classes or methods grow large, which has be en resisted earlier for these small classes, in part to s ave space and keep the code more compact.

When we run this code now, we see all the attributes of our objects, not just the ones we hardcoded in the original repr. And our final issue is resolved: because AttrDisplay takes class names off the self instance dir ectly, each object is shown with the name of its close st (lowest) class—pat displays as a Manager now, not a Person, and we can finally verify that job name is co

rrectly filled in by the Manager constructor:

$ python3 person14.py [Person: job=None, name=Bob Smith, pay=0] [Person: job=dev, name=Sue Jones, pay=100000] Smith Jones [Person: job=dev, name=Sue Jones, pay=110000] Jones [Manager: job=mgr, name=Pat Jones, pay=60000]

This is the more useful display we were after. From a larger perspective, though, our attribute display class has become a general tool, which we can mix into any class by inheritance to leverage the display format it defines. Further, all its clients will automatically pick up future changes in our tool. Later in the book, we'll explore even more powerful class tool concepts, such as decorators and metaclasses; along with Python's many introspection tools, they allow us to write code that augments and manages classes in structured and maintainable ways.

中文翻译

现在，要在我们的类里使用上一节的通用显示工具，我们需要做的只是：从它的模块导入它，通过继承把它混入我们的顶层类，并去掉之前编写的更具体的 repr。这个新的显示重载方法会被 Person 的实例继承，Manager 也一样；Manager 从 Person 获得 repr，而 Person 现在从写在另一个模块里的 AttrDisplay 获得它。Example 28-14 编写了应用这些改动后的 person.py 最终版本，并导入使用独立的 Example 28-13。因为这是最终修订版，我们在这里添加了一些注释来记录我们的工作——用 docstring 做功能

描述、用 # 做小注释，遵循最佳实践惯例；方法之间也加了空行提高可读性——当类或方法变大时这通常是不错的风格选择，之前为节省篇幅、保持代码紧凑，对这些小类一直克制着没这么做。现在运行这段代码，我们能看到对象的所有属性，而不只是最初 repr 里硬编码的那些。而且我们最后的问题也解决了：因为 AttrDisplay 直接取自 self 实例的类名，每个对象都显示它最近（最低）的类的名字——pat 现在显示为 Manager 而不是 Person，我们终于可以验证 job 名被 Manager 构造函数正确填入了： $ python3 person14.py [Person: job=None, name=Bob Smith, pay=0] [Person: job=dev, name=Sue Jones, pay=100000] Smith Jones [Person: job=dev, name=Sue Jones, pay=110000] Jones [Manager: job=mgr, name=Pat Jones, pay=60000] 这就是我们想要的更有用的显示。不过从更大的视角看，我们的属性显示类已经成为一个通用工具，我们可以通过继承把它混入任何类，利用它定义的显示格式。而且，它的所有客户都会自动获得我们工具未来的改动。在本书后面，我们将探索更强大的类工具概念，比如装饰器和元类；连同 Python 的许多内省工具，它们让我们能以结构化、可维护的方式编写增强和管理类的代码。

深度理解

·核心概念：最终形态 = 继承链 AttrDisplay → Person → Manager。显示职责被"外包"给通用工具类，业务类只管数据与业务逻辑。这是本章所有技巧的大汇总：构造、方法、重载、继承、内省。

·底层实现：pat 的 repr 查找路径：Manager（无）→ Person（无）→ AttrDisplay（有，命中）。AttrDisplay

的 repr 用 self.class.name 取到'Manager'，用 self.dict 列出 job/name/pay 全部三个属性。注释和 docstring 体现"文档化代码"的最佳实践。

·设计原因：Person(AttrDisplay) 是一次"混入"（mixin）——Person 不"是"显示工具，但"拥有"显示能力。通用工具与业务代码解耦：改显示格式只动 classtools.py，所有客户自动跟随。

·实际问题：输出现在验证了三个此前无法验证的事实：job 字段确为'mgr'（Manager 构造正确）、pay 为 60000（多态涨薪正确）、pat 显示为 Manager（类名真实）。

·初学者误区：以为"显示工具"和"业务类"必须写在一起；其实跨模块混入正是 Python 组合与复用文化的一部分。注意 from classtools import AttrDisplay 展示了类与模块的关系：类是模块里的名字，通过 import 复用。

代码分析

```
from classtools import AttrDisplay        #  = …

class Person(AttrDisplay):                # Person  MRO
                                          # [Person, AttrDisplay, …
    def __init__(self, name, job=None, pay=0):
        self.name = name
        self.job = job
        self.pay = pay
    ...

class Manager(Person):                    # MRO: [Manager, Person,…
    ...
```

```
pat = Manager('Pat Jones', 50000)          # +
print(pat)                                 #  AttrDisplay
# : [Manager: job=mgr, name=Pat Jones, pay=60000]
```

·解释器如何处理：Manager('Pat Jones', 50000) → Manager 的 init → Person.init(self, name,'mgr', pay) 挂上三个属性。print(pat) → 无 str → 沿 MRO 找 repr，在 AttrDisplay 命中 → self.class.name 动态求值得到'Manager' → gatherAttrs() 遍历 pat 的 dict 排序拼串。全过程中业务类没有写一行显示代码。

·设计思想：关注点分离（separation of concerns）——"是什么"（数据）与"怎么显示"（呈现）分属不同类；通过继承链组合出完整能力。这是装饰器/元类等更强大"类工具"的前奏。

Step 7 (Final): Storing Objects in a Database（第七步（最终）：把对象存入数据库）

英文原文

> At this point, our work is almost complete. We now have a two-module system that not only implements our original design goals for representing people but also provides a general attribute display tool we can use in other programs in the future. By coding functions and classes in module files, we've ensured that they naturally support reuse. And by coding our software as classes, we've ensured that it naturally supports

extension. Although our classes work as planned, though, the objects they create are not real database records. That is, if we kill Python or our program, our instances will disappear—they're transient objects in memory and are not stored in a more permanent medium like a file, so they won't be available in future program runs. It turns out that it's easy to make instance objects more permanent, with a Python feature called object persistence—making objects live on after the program that creates them exits. As a final step in this tutorial, let's make our objects permanent.

中文翻译

此时，我们的工作几乎完成了。我们现在有了一个双模块系统：它不仅实现了我们"表示人"的原始设计目标，还提供了一个将来可以在其他程序里使用的通用属性显示工具。通过把函数和类写进模块文件，我们确保了它们天然支持复用（reuse）。通过把软件编写成类，我们确保了它天然支持扩展（extension）。不过，虽然我们的类按计划工作，但它们创建的对象还不是真正的数据库记录。也就是说，如果我们杀掉 Python 或我们的程序，实例就会消失——它们是内存中的临时对象，没有存储到文件这样的更永久介质里，所以在未来的程序运行中无法再用。事实证明，用一个名为对象持久化（object persistence）的 Python 特性，让实例对象变得更持久是很容易的——让对象在创建它们的程序退出后继续存活。作为本教程的最后一步，让我们把对象变得永久。

深度理解

·核心概念：对象持久化（object persistence）——把内存对象保存到磁盘，程序退出后对象依然存在。本章的收尾：把"记录 + 程序"的类实例变成真正可长期保存的数据库记录。

·底层实现：内存对象是进程内的临时数据，进程结束即被操作系统回收；持久化的本质是"把对象序列化到文件，下次反序列化回来"。Python 的序列化标准工具是 pickle。

·设计原因：为什么前面几步都在"类"里打转？因为数据（属性）+ 行为（方法）被打包后，序列化才有价值——存进去的是完整的"会走路的记录"。

·实际问题：真实软件几乎都离不开持久化：配置、缓存、数据库。本章用最轻量的 shelve 方案展示全流程，为后面接触 SQL/JSON 等留下接口。

·初学者误区：以为"保存对象 = 保存变量"——程序结束时变量就没了；必须显式序列化到文件。也常有人误以为 pickle 的字节流是给人读的文本——不是，它是二进制协议。

Pickles and Shelves（Pickle 与 Shelve）

英文原文

Object persistence is implemented by three standard -library modules, installed with Python itself:- pickle —Serializes arbitrary Python objects to and from a str

ing of bytes- dbm—Implements an access-by-key file system for storing strings- shelve—Uses the other two modules to store Python objects on a file by key We used pickle and noted shelve briefly in Chapter 9 when we studied file basics. They provide simple yet useful data-storage options. Although we can't do them complete justice in this tutorial or book, they are straightforward enough that a brief introduction should suffice to get you started.

英文原文（续）

The pickle module is a general formatting and deformatting tool: given a nearly arbitrary Python object in memory, it converts the object to a string of bytes, which it can use later to reconstruct the original object in memory. The pickle module can handle most Python objects—including lists, dictionaries, nested combinations thereof, and class instances. The latter are especially useful things to pickle because they provide both data (attributes) and behavior (methods); the combination is roughly equivalent to "records" and "programs." Because pickle is so general, it can replace extra code you might otherwise write to create and parse custom text-file representations for your objects. By storing an object's pickle string on a file, you effectively make it persistent: simply load and unpickle it later to re-create the original object.

英文原文（续）

Although it's easy to use pickle by itself to store objects in simple flat files and load them from there later, the shelve module provides an extra layer of structure that allows you to store pickled objects by key. shelve translates an object to its pickled string with pickle and stores that string under a key in a dbm file; when later loading, shelve fetches the pickled string by key and re-creates the original object in memory with pickle. To your script a shelf of pickled objects looks like a dictionary—you index by key to fetch, assign to keys to store, and use dictionary tools such as len, in, and dict.keys to get information. Shelves automatically map most dictionary operations to pickled objects stored in a file. In fact, to your script, the main coding difference between shelves and normal dictionaries is that you must open shelves initially and must close them after making changes. The net effect is that shelve provides a simple database for storing and fetching native Python objects by keys, and thus makes them persistent across program runs. It does not support query tools such as SQL (but Python's sqlite3 standard-library module does) and does not have some advanced features found in enterprise-level databases such as true transaction processing (but other Python database tools do). It does, however, store native Python objects that may be processed with the full power of the Python language once they are fetched

back by key.

中文翻译

对象持久化由三个随 Python 一起安装的标准库模块实现：- pickle——把任意 Python 对象序列化成字节串，并能反序列化回来- dbm——实现一个按键访问（access-by-key）的、用于存储字符串的文件系统- shelve——利用另外两个模块，按键把 Python 对象存储到文件里我们在第 9 章学习文件基础时用过 pickle，也简要提到过 shelve。它们提供了简单而实用的数据存储选项。虽然在本教程或本书中无法对它们做完整论述，但它们足够直白，简短的介绍就足以让你起步了。pickle 模块是一个通用的格式化和反格式化工具：给定内存中一个几乎任意的 Python 对象，它把对象转换成字节串，之后可以用这个字节串在内存中重建原始对象。pickle 模块能处理大多数 Python 对象——包括列表、字典、它们的嵌套组合，以及类实例。后者尤其值得 pickle，因为它们既提供数据（属性）又提供行为（方法）；这种组合大致相当于"记录"和"程序"。因为 pickle 如此通用，它可以替代你原本可能为对象编写"自定义文本文件表示法"及其解析的额外代码。把对象的 pickle 字符串存到文件里，你就有效地让它持久化了：之后只需加载并反 pickle 就能重新创建原始对象。虽然单独使用 pickle 把对象存入简单平面文件、之后再从中加载很容易，但 shelve 模块提供了额外一层结构，允许你按键（key）存储 pickle 后的对象。shelve 用 pickle 把对象翻译成 pickle 字符串，并把该字符串作为一个键对应的值存进 dbm 文件；稍后加载时，shelve 按键取回 pickle 字符串，再用 pickle 在内存中重建原始对象。对你的脚本来说，一个装满 pickle 对

象的 shelf 看起来就像一个字典——按键索引来取、给键赋值来存，用 len、in、dict.keys 等字典工具获取信息。shelf 会自动把大多数字典操作映射到存储在文件里的 pickle 对象上。事实上，对你的脚本而言，shelf 与普通字典的主要编码区别是：你必须先 open 打开 shelf，做修改后还必须 close 关闭它。净效果是：shelve 提供了一个简单的数据库，用来按键存储和获取原生 Python 对象，从而让它们跨程序运行保持持久。它不支持 SQL 这样的查询工具（但 Python 的 sqlite3 标准库模块支持），也没有企业级数据库的一些高级特性，比如真正的事务处理（但其他 Python 数据库工具支持）。不过，它的确能存储原生 Python 对象，一旦按键取回，就能用 Python 语言的全部威力处理它们。

深度理解

·核心概念：三层持久化架构——pickle（序列化）→ dbm（按键存取）→ shelve（字典式数据库界面）。shelve 对程序员呈现的接口就是"字典"。

·底层实现：shelve.open() 返回一个 Shelf 对象，db[key] = obj 时内部调用 pickle.dumps(obj) 得到字节串，再写入 dbm 文件（哈希文件，内容不可读）；db[key] 时从文件取字节串，pickle.loads 重建对象。写后必须 close()，因为改动需要落盘。

·设计原因：为什么类实例特别适合 pickle？因为 pickle 保存"属性 + 类引用"，反序列化后自动重新挂上行为——"记录"与"程序"一起复活。相比手写文本格式（如 CSV/JSON 手工解析），pickle 免去全部解析代码。

·实际问题：shelve 是"够用就好"的数据库：无需服务器、无需 SQL，适合个人数据库、程序配置；但无查询语言、无事务。需要这些时换 sqlite3 或专业数据库。

·初学者误区：① 以为 shelf 文件可以当文本打开阅读——它是二进制哈希文件；② 忘记 close()，改动丢失；③ 以为能 pickle 一切——文件句柄、套接字、lambda 等不可 pickle。

Storing Objects on a shelve Database（把对象存进 shelve 数据库）

英文原文

Pickles and shelves are somewhat advanced topics, and we won't go into all their details here; you can read more about them in the standard-library manuals. This is all simpler in code than narrative, though, so let's jump right in. First up is a new script to store objects of our classes on a shelf. Since this is a new file, we'll need to import our classes in order to make instances to store. We used from to load a class earlier, but as noted in the prior chapter, there are two ways to get a class from a file (class names are variables like any other, and not at all magic in this context). As an abstract refresher: python import person # Load class with import bob = person.Person(...) # Go through module name from person import Person # Load class with from bob = Person(...) # Use name directly We'

ll use from to load in our script, just because it's less to type. To also keep this simple, we'll duplicate som e of the self-test lines from person.py that make insta nces of our classes, so we have something to store (w e won't fret over the test-code redundancy in this si mple demo). Once we have some instances, it's almo st trivial to store them on a shelf. We simply import t he shelve module, open a new shelf with an external filename, assign the objects to keys in the shelf, and close the shelf when we're done because we've made changes—all per Example 28-15.

Example 28-15: makedb.py（把 Person 对象存进 shelve 数据库）

```
from person_14 import Person, Manager    #
bob = Person('Bob Smith')                #
sue = Person('Sue Jones', job='dev', pay=100000)
pat = Manager('Pat Jones', 50000)
import shelve
db = shelve.open('persondb')             #
for obj in (bob, sue, pat):              #  name
    db[obj.name] = obj                   #  shelf
db.close()                               #
```

Notice how we assign objects to the shelf using their own names as keys. This is just for convenience; in a shelf, the key can be any string, including one we mi ght create to be unique using tools such as process I Ds and timestamps (available in the os and time stan dard-library modules). The only rule is that the keys must be strings and should be unique since we can st

ore just one object per key—though that object can be a list, dictionary, or other object containing many objects itself. In fact, the values we store under keys can be Python objects of almost any sort: built-in types like strings, lists, tuples, and dictionaries, as well as user-defined class instances, and nested combinations of all of these and more. For example, the name and job attributes of our objects could be nested dictionaries and lists as in earlier incarnations in this book (though this would require a bit of redesign to the current code). That's all there is to it—if this script has no output when run, it means it probably worked; we're not printing anything, just creating and storing objects in a file-based database: $ python3 makedb.py

中文翻译

pickle 和 shelf 是有点高级的话题，我们不会在这里深入所有细节；你可以去标准库手册里读更多。不过，这些用代码表达比叙述更简单，所以我们直接开干。首先是一个新脚本，把我们的类对象存到 shelf 上。因为这是一个新文件，我们需要导入我们的类才能创建要存储的实例。我们前面用过 from 加载类，但正如前一章所说，从文件获取类有两种方式（类名和其他任何变量一样，在这个上下文中毫无魔力）。作为抽象的复习：python import person # 用 import 加载类bob = person.Person(...) # 通过模块名访问from person import Person # 用 from 加载类bob = Person(...) # 直接使用名字我们脚本里用 from 加载，只是因为它打字少。为了保持简单，我们会从 person.py 里复制一些

创建类实例的自测行，以便有东西可存（在这个简单演示里，我们不为测试代码的冗余纠结）。一旦有了实例，把它们存进 shelf 就几乎是微不足道的事：我们只需导入 shelve 模块，用一个外部文件名打开新 shelf，把对象赋给 shelf 里的键，做完修改后关闭 shelf——一切都按 Example 28-15。注意我们是如何用对象自己的名字作为键把它们赋给 shelf 的。这只是为了方便；在 shelf 里，键可以是任何字符串，包括我们用进程 ID、时间戳（os 和 time 标准库模块里有）等工具造出来的唯一字符串。唯一的规则是：键必须是字符串，并且应该唯一，因为每个键只能存一个对象——尽管这个对象本身可以是包含许多对象的列表、字典或其他对象。事实上，我们存在键下的值几乎可以是任何种类的 Python 对象：字符串、列表、元组、字典等内置类型，以及用户自定义的类实例，还有所有这些甚至更多的嵌套组合。例如，我们对象的 name 和 job 属性可以是嵌套的字典和列表，就像本书前面章节的早期版本那样（不过这需要对当前代码稍作重新设计）。就这些了——如果这个脚本运行后没有输出，说明它很可能成功了；我们什么也没打印，只是创建对象并把它们存进基于文件的数据库： $ python3 makedb.py

深度理解

·核心概念：写库脚本（makedb.py）——创建实例 → 打开 shelf → 按键存储 → 关闭。数据库的"写入端"只有 4 行关键代码。

·底层实现：db[obj.name] = obj 触发 Shelf 的 setitem：pickle.dumps(obj) 序列化（对类实例会记录"类名 + 模块名"），写入 persondb 的 dbm 哈希文件。键必须

是字符串（dbm 的限制）；值可以几乎任意。

·设计原因：用名字当键纯为演示方便；生产环境常用"id"、时间戳、UUID 保证唯一。shelve 的最小接口（open/赋值/close）把学习成本降到极低。

·实际问题：两个导入姿势（import person vs from person import Person）展示"类名是模块命名空间的变量"——再次呼应"类与模块"的关系。

·初学者误区：① 以为 makedb 运行后"什么都没发生"就是失败了——恰恰相反，静默成功；② 忘关 shelf 导致缓存未落盘。

代码分析

```
from person_14 import Person, Manager    #
bob = Person('Bob Smith')                # person_14
import shelve
db = shelve.open('persondb')             #  persondb  sh…
for obj in (bob, sue, pat):
    db[obj.name] = obj                   # obj.name
db.close()                               #
```

·解释器如何处理：from person14 import Person, Manager 导入时执行 person14 模块的顶层代码（定义类；因 name 不是'main'，跳过测试体）。shelve.open('persondb') 打开 dbm 数据库；循环里每次赋值都对实例做 pickle 序列化（记录属性和类的模块/名字），然后写入哈希文件；db.close() 把缓冲区写回磁盘。

·设计思想：把"写库"和"读库"分成独立脚本，保持每个脚本职责单一；数据库文件（persondb）成为程序间传递数据的媒介——这就是"对象持久化"的意义。

Exploring Shelves Interactively（交互式探索 shelf)

英文原文

> At this point, there are one or more files in the current directory whose names all start with persondb. The actual files created can vary, and just as in the built-in open function, the filename in shelve.open() is relative to the current working directory (CWD) unless it includes a directory path.

> Wherever they are stored, these files implement a keyed-access file that contains the pickled representation of our three Python objects. Don't delete these files—they are your database and are what you'll need to copy or transfer when you back up or move your storage.

> You can inspect the shelf's files either from a file explorer, console shell, or Python REPL, but they are binary hash files, and most of their content makes little sense outside the context of the shelve module. For example, Python's standard-library glob module allows us to get directory listings to verify the shelf (it's just one file on macOS, persondb.db), and we can open it in binary mode to explore stored bytes:

>> import glob >> glob.glob('persondb') ['persondb.db'] >> print(open('persondb.db','rb').read()) b'\x0

0\x06\x15a\x00\x00\x00\x02\x00\x00\x04\xd2\x00\x00... much more omitted...

This content can vary, but it's nearly impossible to decipher here, and doesn't exactly qualify as a user-friendly database interface. To verify better, we can write another script, or poke around our shelf at the interactive prompt. Because shelves are Python objects containing Python objects, we can process them with normal Python syntax and development modes. Here, the REPL effectively becomes a database client:

$ python3 >> import shelve >> db = shelve.open('persondb') # Reopen the shelf >> len(db) # Three objects stored 3 >> list(db.keys()) # keys is the index ['Bob Smith','Sue Jones','Pat Jones'] >> pat = db['Pat Jones'] # Fetch an object by key >> pat.lastName() # Runs lastName from Person'Jones'>> pat # Runs repr from AttrDisplay [Manager: job=mgr, name=Pat Jones, pay=50000] >> for key in sorted(db): # Iterate, sort, fetch, print

print(key,'=>', db[key]) Bob Smith => [Person: job=None, name=Bob Smith, pay=0] Pat Jones => [Manager: job=mgr, name=Pat Jones, pay=50000] Sue Jones => [Person: job=dev, name=Sue Jones, pay=100000]

Notice that we don't have to import our Person or Manager classes here in order to load or use our stored objects. For example, we can call pat's lastName method freely, and get its custom print display format au

tomatically, even though we don't have the Person cl ass in scope here.

> This works because when Python pickles a class insta nce, it records the instance's self attributes, along wit h the names of the class it was created from and the module where that class lives. When an instance is lat er fetched from the shelf and unpickled, Python auto matically reimports the class by name and makes a n ew instance of it having the previously stored instanc e attributes.
>
> The upshot of this scheme is that class instances auto matically acquire all their class behavior when they ar e loaded in the future. We have to import our classes only to make new instances, not to process existing o nes. Although a deliberate feature, this scheme has s omewhat mixed consequences: - The downside is tha t classes and their module's files must be importable when an instance is later loaded.
>
> That is, picklable classes must be coded at the top le vel of a module file that is accessible from a directory listed on the sys.path module search path (and shoul dn't live in the topmost script files' module main unle ss they're always in that module when used).

Because of this external file requirement, some progr ams pickle simpler objects such as dictionaries or list s, especially if they are to be transferred across netw orks. - The upside is that changes in a class's source c

ode file are automatically picked up when instances of the class are loaded again. There is often no need to update stored objects themselves since updating their class's code changes their behavior.

Shelves have other well-known limitations covered in Python's library manual. For simple object storage, though, shelves and pickles are easy-to-use tools for personal databases, program configurations, and more.

中文翻译

此时，当前目录里会出现一个或多个文件名都以 persondb 开头的文件。实际创建的文件可能有所不同；与内置 open 函数一样，shelve.open() 里的文件名相对于当前工作目录（CWD），除非它包含目录路径。无论存在哪里，这些文件实现了一个按键访问的文件，里面装着我们的三个 Python 对象的 pickle 表示。别删这些文件——它们是你的数据库，备份或迁移存储时需要复制或传输的正是它们。你可以用文件资源管理器、控制台 shell 或 Python REPL 检查 shelf 的文件，但它们是二进制哈希文件，大多数内容在 shelve 模块的语境之外没什么意义。例如，Python 标准库的 glob 模块可以帮我们列出目录以确认 shelf 的存在（在 macOS 上它只是一个文件，persondb.db），我们还可以用二进制模式打开它，查看存储的字节： >>> import glob >>> glob.glob('persondb') ['persondb.db'] >>> print(open('persondb.db','rb').read()) b'\x00\x06\x15a\x00\x00\x00\x02\x00\x00\x04\xd2\x00\x00...much more omitted...（大量内容省略）这些内容可能各不相同，但在这

里几乎不可能解读，也算不上用户友好的数据库界面。要更好地验证，我们可以再写一个脚本，或者在交互提示符下翻看我们的 shelf。因为 shelf 是装着 Python 对象的 Python 对象，我们可以用普通的 Python 语法和开发模式处理它们。在这里，REPL 实际上变成了一个数据库客户端（database client）： $ python3 >>> import shelve >>> db = shelve.open('persondb') # 重新打开 shelf >>> len(db) # 存了三个对象3 >>> list(db.keys()) # keys 就是索引['Bob Smith','Sue Jones','Pat Jones'] >>> pat = db['Pat Jones'] # 按键取对象>>> pat.lastName() # 运行来自 Person 的 lastName'Jones'>>> pat # 运行来自 AttrDisplay 的 repr [Manager: job=mgr, name=Pat Jones, pay=50000] >>> for key in sorted(db): # 迭代、排序、取、打印print(key,'=>', db[key]) Bob Smith => [Person: job=None, name=Bob Smith, pay=0] Pat Jones => [Manager: job=mgr, name=Pat Jones, pay=50000] Sue Jones => [Person: job=dev, name=Sue Jones, pay=100000] 注意：要加载或使用我们存储的对象，这里根本不需要导入 Person 或 Manager 类。例如，我们可以随意调用 pat 的 lastName 方法，自动获得它的自定义打印显示格式，即使 Person 类根本不在当前作用域里。这之所以有效，是因为 Python 在 pickle 一个类实例时，会记录实例的 self 属性，以及"创建它的类"的名字和那个类所在的模块名。之后从 shelf 取出实例并反 pickle 时，Python 会自动按名字重新导入这个类，并创建一个拥有先前存储的实例属性的新实例。这种方案的结果是：类实例将来被加载时，会自动获得其类的全部行为。我们只需要在创建新实例时导入类，处理已有实例则不需要。虽然这是个深思熟虑的特

性，但该方案有着好坏参半的后果：- 坏处是：实例日后加载时，类及其模块文件必须可导入（importable）。也就是说，可 pickle 的类必须写在模块文件的顶层，并且该模块可从 sys.path 模块搜索路径列出的目录访问（而且除非使用时总是位于那个模块中，否则不应放在最顶层脚本文件的 main 模块里）。由于这个外部文件需求，有些程序会 pickle 更简单的对象，比如字典或列表，尤其是要跨网络传输时。- 好处是：类的源代码文件一旦修改，类的实例再次加载时会自动采用新代码。通常不需要更新存储的对象本身，因为更新类的代码就改变了它们的行为。shelf 还有其他众所周知的局限，Python 的库手册里都有说明。不过，就简单的对象存储而言，shelf 和 pickle 是个人数据库、程序配置等场景中易用的工具。

深度理解

·核心概念：pickle 的"类自愈"机制——反序列化实例时，pickle 记录的不是对象本体，而是"属性 + 类名 + 模块名"；加载时自动 import 该类并重建实例。所以读库脚本无需导入 Person/Manager。

·底层实现：db['Pat Jones'] → 从 dbm 取字节串 → pickle.loads 解析：找到 person14.Person（或 Manager）的引用 → 动态导入模块 → 创建新实例 → 恢复属性。pat.lastName()、pat 的显示因此自动可用——多态与运算符重载在"数据库外"依然生效。

·设计原因：这个设计是故意的：行为（方法）不随数据存储，而靠类代码"按需加载"——磁盘只存数据，代码永远最新。缺点是强依赖"类必须可导入"（sys.path 可达、不

在 main）。

·实际问题：REPL 即数据库客户端——len(db)、db.keys()、db[key]、sorted(db) 与字典如出一辙；二进制字节（b'\x00\x06...'）则提醒我们：shelf 文件不是给人读的文本。

·初学者误区：① 以为反序列化"复制了类代码"——不，它导入的是你的模块；删掉 person14.py 后实例将无法加载；② 把类定义写在脚本的 main 块里导致无法被按名导入——pickle 会失败；③ 以为 db.keys() 返回列表——它是视图，需要 list() 包装（Python 3 行为）。

代码分析

```
import shelve
db = shelve.open('persondb')      #
pat = db['Pat Jones']             #  pickle  Manager
pat.lastName()                    # 'Jones'—
print(pat)                        # [Manager: job=mgr, name=Pat Jo…
for key in sorted(db):            #
    print(key, '=>', db[key])     #  db[key]
```

·解释器如何处理：db['Pat Jones'] 调用 Shelf 的 getitem：按键从 dbm 取 pickle 字节串，交给 pickle.loads；解析出类引用 person14.Manager 后，解释器执行一次模块导入（若尚未导入），再构造新 Manager 实例并写入存储的属性。因此 pat.lastName() 沿 MRO 找到方法执行，print(pat) 沿 MRO 找到 AttrDisplay 的 repr 显示。

·设计思想：数据与代码分离的极致体现——磁盘只有"数据"，代码永远从最新源码来。这是"面向对象数据库"思想

的迷你版本：对象跨进程、跨运行存活。

Updating Objects on a Shelf（更新 shelf 上的对象）

英文原文

> One last script: let's write a program that updates an instance (record) each time it runs, to show that our objects really are persistent—that their current values are available every time a Python program runs. The file coded in Example 28-16 prints the database and gives a raise to one of our stored objects on each run. If you trace through what's going on here, you'll notice that we're getting a lot of utility "for free"—printing our objects automatically employs the general repr overloading method, and we give raises by calling the giveRaise method we wrote earlier. This all just works for objects based on OOP's inheritance model, even when they live in a file.

Example 28-16: updatedb.py（修改 shelve 数据库中的 Person 对象）

```
import shelve
db = shelve.open('persondb')            #  shelf
for key in sorted(db):                  #
    print(key, '\t=>', db[key])         #
sue = db['Sue Jones']                   #
sue.giveRaise(.10)                      #
db['Sue Jones'] = sue                   #  shelf
db.close()                              #
```

Because this script prints the database when it starts up, we have to run it at least twice to see our objects change. Here it is in action, displaying all records and increasing sue's pay each time it is run (it's a pretty g ood script for sue; something to schedule to run regu larly as a cron job perhaps?):

$ python3 updatedb.py Bob Smith => [Person: job= None, name=Bob Smith, pay=0] Pat Jones => [Mana ger: job=mgr, name=Pat Jones, pay=50000] Sue Jone s => [Person: job=dev, name=Sue Jones, pay=10000 0]

$ python3 updatedb.py Bob Smith => [Person: job= None, name=Bob Smith, pay=0] Pat Jones => [Mana ger: job=mgr, name=Pat Jones, pay=50000] Sue Jone s => [Person: job=dev, name=Sue Jones, pay=11000 0]

$ python3 updatedb.py Bob Smith => [Person: job= None, name=Bob Smith, pay=0] Pat Jones => [Mana ger: job=mgr, name=Pat Jones, pay=50000] Sue Jone s => [Person: job=dev, name=Sue Jones, pay=12100 0]

Again, what we see here is a product of the shelve and pickle tools we get from Python, and of the behavior we coded in our classes ourselves. And once again, we can verify our script's work at the interactive prompt—the shelf's equivalent of a database client:

$ python3 >> import shelve >> db = shelve.open('persondb') # Reopen database >> rec = db['Sue Jones'] # Fetch object by key >> rec [Person: job=dev, name=Sue Jones, pay=133100] >> rec.lastName(), rec.pay ('Jones', 133100)

For another example of object persistence in this book, watch for the sidebar "Why You Will Care: Classes and Persistence". It stores a somewhat larger composite object in a flat file with pickle instead of shelve, but the effect is similar. For more details and examples of pickles, see also Chapter 9 (file basics) and Chapter 37 (string tools and Unicode), as well as Python's manuals.

中文翻译

最后一个脚本：让我们写一个每次运行都更新一个实例（记录）的程序，以证明我们的对象真的持久——它们的当前值在每次 Python 程序运行时都可用。Example 28-16 编写的文件每次运行都会打印数据库，并给其中一个存储对象涨薪。如果你跟踪这里发生的事，会注意到我们"免费"获得了很多工具价值——打印对象自动使用通用的 repr 重载方法，涨薪则调用我们早先编写的 giveRaise 方法。所有这一切对基于 OOP 继承模型的对象都直接生效，哪怕它们活

在文件里。因为这个脚本启动时会打印数据库，我们必须至少运行两次才能看到对象变化。看它的实际运行——每次运行都显示所有记录并给 sue 涨薪（对 sue 来说这是个相当不错的脚本；也许可以安排它定期作为 cron 任务运行？）： $ python3 updatedb.py Bob Smith => [Person: job=None, name=Bob Smith, pay=0] Pat Jones => [Manager: job=mgr, name=Pat Jones, pay=50000] Sue Jones => [Person: job=dev, name=Sue Jones, pay=100000] $ python3 updatedb.py Bob Smith => [Person: job=None, name=Bob Smith, pay=0] Pat Jones => [Manager: job=mgr, name=Pat Jones, pay=50000] Sue Jones => [Person: job=dev, name=Sue Jones, pay=110000] $ python3 updatedb.py Bob Smith => [Person: job=None, name=Bob Smith, pay=0] Pat Jones => [Manager: job=mgr, name=Pat Jones, pay=50000] Sue Jones => [Person: job=dev, name=Sue Jones, pay=121000] 再说一次，我们在这里看到的，是 Python 提供的 shelve 和 pickle 工具，加上我们自己写在类里的行为共同作用的结果。再一次，我们可以在交互提示符下验证脚本的工作——即 shelf 版的数据库客户端： $ python3 >>> import shelve >>> db = shelve.open('persondb') # 重新打开数据库>>> rec = db['Sue Jones'] # 按键取对象>>> rec [Person: job=dev, name=Sue Jones, pay=133100] >>> rec.lastName(), rec.pay ('Jones', 133100) 关于本书中对象持久化的另一个例子，请注意侧栏 "Why You Will Care: Classes and Persistence"（为什么你会关心：类与持久化）。它把一个大一些的复合对象用 pickle 存进平面文件而不是 shelve，但效果类似。关于 pickle 的更多

细节和示例，还可以参见第 9 章（文件基础）和第 37 章（字符串工具与 Unicode），以及 Python 的手册。

深度理解

·核心概念："读-改-写"三步更新模式——取对象（db['Sue Jones']）→ 内存中修改（sue.giveRaise(.10)）→ 写回（db['Sue Jones'] = sue）。这正是 shelve 与真实数据库（如 SQL）更新模式的核心差异：shelve 更新的是"整个对象"，没有 UPDATE 语句。

·底层实现：giveRaise 在内存里把 sue 的 pay 从 100000 改到 110000（int(1000001.1)）；db['Sue Jones'] = sue 再次 pickle 整个对象覆盖旧值；close() 落盘。每次运行 pay 按 1.1 累积：100000 → 110000 → 121000 → 133100。

·设计原因：这个脚本同时演示了"多态、重载、继承、持久化"四者的叠加效应——一行 print(key,'=>', db[key]) 的显示逻辑来自 AttrDisplay，涨薪行为来自继承链上的方法，而数据跨运行存活。

·实际问题：这是"定时任务/批处理"的雏形（cron 定期跑涨薪）；也是备份/迁移的提醒：persondb 文件就是整个数据库。

·初学者误区：① 改了对象却忘了写回键（db['Sue Jones'] = sue），下次运行发现没变化——shelve 不会自动同步内存对象；② 以为 giveRaise 会直接改文件——不，它只改内存里的对象。

代码分析

```
import shelve
db = shelve.open('persondb')            #
for key in sorted(db):                  #
    print(key, '\t=>', db[key])
sue = db['Sue Jones']                   # 1.  Person
sue.giveRaise(.10)                      # 2. pay *= 1.1
db['Sue Jones'] = sue                   # 3.
db.close()                              #
```

·解释器如何处理：每次 python3 updatedb.py 都是一次全新的进程：打开文件 → 反序列化全部记录打印 → 对 Sue 的副本执行 giveRaise(.10)（pay 从上次存储值继续累乘 1.1）→ 序列化覆盖存储 → 关闭。第二次运行打印的 110000、第三次 121000，证明状态跨进程保留——这就是"持久化"的直接证据。

·设计思想：程序 = 处理数据库的工具（客户端），数据库文件 = 状态的唯一真相源。OOP 三特性 + 持久化在此完美闭环。

Future Directions（未来方向）

英文原文

> And that's a wrap for this chapter's tutorial. At this point, you've seen all the basics of Python's OOP machinery in action, and you've learned ways to avoid redundancy and its associated maintenance issues in your code. You've built full-featured classes that do real work. As an added bonus, you've made them real dat

abase records by storing them in a Python shelf, so t heir information lives on persistently. There is much more we could explore here, of course. For example, we could extend our classes to make them more reali stic, add new kinds of behavior to them, and so on. G iving a raise, for instance, should in practice verify th at pay increase rates are between zero and one—an extension we'll add when we meet decorators later in this book. You might also mutate this example into a personal contacts database, by changing the state inf ormation stored on objects, as well as the classes' me thods used to process it. We'll leave this a suggested exercise open to your imagination. We could also exp and our scope to use tools that either come with Pyt hon or are freely available in the open source world. For instance, GUIs, websites, or apps would make our database more accessible to nonprogrammers, and o ther database interfaces like sqlite3 and content-repr esentation schemes like JSON offer additional possibi lities. While this chapter has hopefully sparked your i nterest for future exploration, such topics are of cour se beyond the scope of this tutorial and this book at l arge. If you want to explore any of them on your own , see the web, Python's standard-library manuals, and follow-up application texts. First, though, let's return to class fundamentals and finish up the rest of the Py thon core-language story.

中文翻译

> 本教程到此告一段落。此时，你已经看到了 Python OOP 机制的全部基础在行动，也学会了避免代码冗余及其相关维护问题的方法。你构建了能做实际工作的功能完整的类。作为额外奖励，你通过把对象存进 Python shelf 让它们成为真正的数据库记录，所以它们的信息可以持久地存活下去。当然，这里还有很多可以探索的。例如，我们可以扩展类让它们更贴近现实、给它们添加新类型的行为，等等。比如，涨薪在实践里应该验证加薪比率在 0 到 1 之间——这是我们在本书后面遇到装饰器（decorators）时要添加的扩展。你也可以把这个例子改造成个人通讯录数据库，方法是修改对象上存储的状态信息，以及处理它们的类方法。我们把这个作为建议练习留给你，展开你的想象吧。我们也可以扩大范围，使用随 Python 自带或开源世界免费提供的工具。例如，GUI、网站或应用可以让我们的数据库对非程序员更友好，而 sqlite3 等其他数据库接口和 JSON 等内容表示方案提供了额外的可能性。希望本章点燃了你未来探索的兴趣，但这类话题当然超出了本教程乃至本书的范畴。如果你想自己探索其中任何一个，请查阅网络、Python 标准库手册和后续的应用类书籍。不过，首先让我们回到类的基础，把 Python 核心语言故事的其余部分讲完。

深度理解

·核心概念：本章是"OOP 基础全览"的收官，作者给出三个未来方向：参数校验（装饰器，第 39 章）、业务改造（改成通讯录）、技术升级（GUI/sqlite3/JSON）。

·底层实现：注意 @rangetest(percent=(0.0, 1.0)) 在 St

ep 2 结尾的预告——装饰器本质是"包装函数的函数"，能把校验逻辑注入方法而不用改方法体。

·设计原因：作者始终贯彻"先给全貌、再讲细节"的教学法：本章结尾没有引入新概念，而是指明"你已掌握基础，接下来是装饰器、元类、设计模式"。

·实际问题：给出改造练习（把 name/job/pay 换成 name/address/birthday/phone/email 等）——这是把"模板代码"迁移到新业务的第一次热身。

·初学者误区：以为"学完本章 = 学完 OOP"——本章是基础全景；类机制还有很多细节（第 29~31 章），设计模式（第 31 章）与高级特性（装饰器/元类）还在后面。

Chapter Summary（本章小结）

英文原文

In this chapter, we explored all the fundamentals of Python classes and OOP in action, by building upon a simple but real example, step by step. We added constructors, methods, operator overloading, customization with subclasses, and introspection-based tools, and we met concepts such as composition, delegation, and polymorphism along the way. In the end, we took objects created by our classes and made them persistent by storing them on a shelve object database—a simple system for saving and retrieving native Python objects by key. While exploring class basics, we also

encountered multiple ways to factor our code to reduce redundancy and minimize future maintenance costs. In the next chapters of this part of the book, we'll resume our study of the details behind Python's class model and investigate its application to some of the design concepts used to combine classes in larger programs. Before we move ahead, though, let's work through this chapter's quiz to review what we covered here. Since we've already done a lot of hands-on work in this chapter, we'll close with a set of mostly theory-oriented questions designed to make you trace through some of the code and ponder some of the bigger ideas behind it.

中文翻译

在本章中，我们以一个小小的但真实的例子为基础，一步步探索了 Python 类与 OOP 的全部基础在行动中的表现。我们添加了构造函数、方法、运算符重载、用子类定制，以及基于内省的工具；一路上还遇到了组合（composition）、委托（delegation）和多态（polymorphism）等概念。最后，我们把自己类的对象存进 shelve 对象数据库——一个按键保存和检索原生 Python 对象的简单系统——让它们持久化。在探索类基础的同时，我们还遇到了多种分解代码的方法，以消除冗余、把未来的维护成本降到最低。在本书这部分接下来的章节里，我们将恢复对 Python 类模型背后细节的研究，并考察它在一部分用于组合大型程序中类的设计概念上的应用。不过，在继续前进之前，让我们先做一下本章的测验，回顾我们所学的内容。因为本章已经做了大

量动手练习，我们最后会用一组偏理论的问题收尾，目的是让你跟踪一些代码并思考它背后更大的思想。

Test Your Knowledge: Quiz（测验）

英文原文

1. When we fetch a Manager object from the shelf and print it, where does the display format logic come from? 2. When we fetch a Person object from a shelf without importing its module, how does the object know that it has a giveRaise method that we can call? 3. Why is it so important to move processing into methods, instead of hardcoding it outside the class? 4. Why is it better to customize by subclassing rather than copying the original and modifying? 5. Why is it better to call back to a superclass method to run default actions, instead of copying and modifying its code in a subclass? 6. Why is it better to use tools like dict that allow objects to be processed generically than to write more custom code for each type of class? 7. In general terms, when might you choose to use object embedding and composition instead of inheritance? 8. What would you have to change if the objects coded in this chapter used a dictionary for names and a list for jobs, as in similar examples earlier in this book? 9. How might you modify the classes in this chapter to implement a personal contacts database in Pyt

hon?

中文翻译

1. 当我们从 shelf 取回一个 Manager 对象并打印它时，显示格式的逻辑来自哪里？2. 当我们不导入其模块就从 shelf 取回一个 Person 对象时，这个对象怎么知道它有我们可以调用的 giveRaise 方法？3. 为什么把处理逻辑移进方法（而不是硬编码在类外面）如此重要？4. 为什么通过子类化定制比复制原类再修改更好？5. 为什么调用超类方法执行默认操作，比在子类里复制并修改它的代码更好？6. 为什么使用 dict 这类允许对象被通用处理的工具，比为每种类编写更多定制代码更好？7. 笼统地说，什么时候你可能会选择对象嵌入和组合，而不是继承？8. 如果本章编写的对象像本书前面类似例子那样，用字典做名字、用列表做工作，你需要改变什么？9. 你会如何修改本章的类，用 Python 实现一个个人通讯录数据库？

Test Your Knowledge: Answers（答案）

英文原文

1. In the final version of our classes, Manager ultimately inherits its repr printing method from AttrDisplay in the separate classtools module, and two levels up in the class tree. Manager doesn't have one itself, so the inheritance search climbs to its Person superclass; because there is no repr there either, the search climbs higher and finds it in AttrDisplay. The class names

listed in parentheses in a class statement's header line provide the links to higher superclasses. 2. Shelves (really, the pickle module they use) automatically relink an instance to the class it was created from when that instance is later loaded back into memory. Python reimports the class from its module internally and creates a new instance with the previously stored attributes. This way, loaded instances automatically obtain all their original methods (like lastName, giveRaise, and repr), even if we have not imported the instance's class into the loading scope. 3. It's important to move processing into methods so that there is only one copy to change in the future, and so that the methods can be run on any instance. This is Python's notion of encapsulation—wrapping up logic behind interfaces, to better support future code maintenance. If you don't do so, you create code redundancy that can multiply your work effort as the code evolves in the future. 4. Customizing with subclasses reduces development effort. In OOP, we code by customizing what has already been done, rather than copying or changing existing code. This is the real "big idea" in OOP—because we can easily extend our prior work by coding new subclasses, we can leverage what we've already done. This is much better than either starting from scratch each time, or introducing multiple redundant copies of code that may all have to be updated in the future. 5. Copying and modifying code doubles your p

otential work effort in the future, regardless of the co ntext. If a subclass method needs to perform default actions coded in a superclass method, it's much bett er to call back to the original through the superclass' s name (or the super built-in) than to copy its code. T his also holds true for superclass constructors. Again, copying code creates redundancy, which is a major is sue as code evolves. 6. Generic tools can avoid hardc oded solutions that must be kept in sync with the res t of the class as it evolves over time. A generic repr p rint method, for example, need not be updated each time a new attribute is added to instances in an init c onstructor. In addition, a generic print method inheri ted by all classes appears and need be modified in o nly one place—changes in the generic version are pic ked up by all classes that inherit from the generic cla ss. Again, eliminating code redundancy cuts future d evelopment effort; that's one of the primary assets cl asses bring to the table. 7. Inheritance is best at codi ng extensions based on direct customization (like our Manager specialization of Person). Composition is we ll suited to scenarios where multiple objects are aggr egated into a whole and directed by a controller-laye r class. Inheritance passes calls up to reuse, and com position passes down to delegate. Inheritance and co mposition are not mutually exclusive; often, the obje cts embedded in a controller are themselves customi zations based upon inheritance. 8. Not much since th

is was really a first-cut prototype, but the lastName method would need to be updated for the new name format; the Person constructor would have to change the job default to an empty list; and the Manager cla ss would probably need to pass along a job list in its constructor instead of a single string (self-test code would change as well, of course). The good news is t hat these changes would need to be made in just on e place—in our classes, where such details are encap sulated. Apart from their object-construction calls, th e database scripts should work as is, as shelves supp ort arbitrarily nested data. 9. The classes in this chapt er could be used as boilerplate "template" code to im plement a variety of types of databases. Essentially, y ou can repurpose them by modifying the constructor s to record different attributes and providing whatev er methods are appropriate for the target application . For instance, you might use attributes such as name , address, birthday, phone, email, and so on for a con tacts database, and methods appropriate for this pur pose. A method named sendmail, for example, might use Python's standard-library smtplib module to sen d an email to one of the contacts automatically when run (see Python's manuals or application-level books for more details on such tools). The AttrDisplay tool we wrote here could be used verbatim to print your o bjects because it is intentionally generic. Most of the shelve database code here can be used to store your

objects, too, with minor changes.

中文翻译

1. 在我们类的最终版本里，Manager 最终从独立的 classtools 模块中的 AttrDisplay 继承了它的 repr 打印方法，这在类树中隔了两层。Manager 自己没有 repr，所以继承查找爬到它的 Person 超类；因为那里也没有 repr，查找继续向上，在 AttrDisplay 里找到它。class 语句头括号里列出的类名，提供了通向更上层超类的链接。2. shelf（准确说是它们使用的 pickle 模块）在实例稍后被重新加载进内存时，会自动把实例重新链接到创建它的类。Python 会在内部从其模块重新导入该类，并用先前存储的属性创建一个新实例。这样，即使我们没有把实例的类导入到加载作用域里，加载出来的实例也会自动获得它们全部原始方法（比如 lastName、giveRaise 和 repr）。3. 把处理逻辑移进方法很重要，这样将来只有一个副本需要修改，而且这些方法可以在任何实例上运行。这就是 Python 的封装（encapsulation）概念——把逻辑包在接口后面，以更好地支持未来的代码维护。如果不这样做，你创造的代码冗余会随着代码演进成倍放大你的工作量。4. 用子类定制能减少开发工作量。在 OOP 中，我们通过定制已完成的工作来编程，而不是复制或修改现有代码。这才是 OOP 真正的"大思想"——因为我们可以通过编写新子类轻松扩展以前的工作，我们就能利用已经做好的东西。这远比"每次从头开始"或"引入多份可能都需要更新的冗余代码副本"要好。5. 无论什么场景，复制并修改代码都会让未来的潜在工作量翻倍。如果子类方法需要执行超类方法里编码的默认操作，最好通过超类的名字（或 super 内置函数）回调原版，而不是复制它的代码

。这对超类构造函数同样成立。再说一次，复制代码制造冗余，而冗余是代码演化的主要问题。6. 通用工具可以避免那些必须随类演化而保持同步的硬编码方案。例如，通用的 repr 打印方法，无需在 init 构造函数每次给实例添加新属性时更新。此外，一个被所有类继承的通用 print 方法只需（也只需）在一处修改——通用版本的改动会被所有继承它的类采纳。再说一次，消除代码冗余削减了未来的开发工作量；这是类带来的主要资产之一。7. 继承最适合编写基于直接定制的扩展（比如我们对 Person 的 Manager 特化）。组合适合"多个对象聚合为一个整体、由控制器层类指挥"的场景。继承把调用传向上去复用，组合把调用传向下去委托。继承和组合并不互斥；通常，控制器里嵌入的对象本身又是基于继承的定制。8. 变化不大，因为这只是初稿原型，但 lastName 方法需要为新名字格式更新；Person 构造函数得把 job 的默认值改成空列表；Manager 类可能需要在构造函数里传递一个 job 列表而不是单个字符串（自测代码当然也要改）。好消息是：这些改动只需要在一处进行——在我们的类里，这些细节都被封装起来了。除了创建对象的调用之外，数据库脚本应该原样可用，因为 shelf 支持任意嵌套的数据。9. 本章的类可以作为样板"模板"代码，用来实现各种类型的数据库。本质上，你可以重新利用它们：修改构造函数以记录不同的属性，并提供适合目标应用的方法。例如，通讯录数据库可以使用 name、address、birthday、phone、email 等属性，以及适合此目的的方法。比如一个名为 sendmail 的方法，运行时可以用 Python 标准库的 smtplib 模块自动给其中一个联系人发邮件（这类工具的更多细节参见 Python 手册或应用层书籍）。我们在这里写的 AttrDisplay 工具可以原样用来打印你的对象

，因为它刻意做得很通用。这里的大部分 shelve 数据库代码稍作修改也可以用来存储你的对象。

原书脚注

1. 当然，绝无冒犯听众中任何经理之意。这个玩笑曾在纽泽西州的一堂 Python 课上讲过，无人发笑。主办方后来透露，当时听课的都是评估 Python 的经理。所以全场沉默。
2. 术语说明：本书现在用名词 "shelf" 指代由模块 shelve 管理的对象存储，用它的复数 "shelves" 指代由 shelve 管理的多个 "shelf"。过去，模块的动词名 "shelve" 也被混淆着当名词用——多年来让本书的编辑（电子的和人类的）非常头疼。

本章总结

技术拓展（Technical Expansion）

·实际项目中的应用场景：本章的 Person/Manager + shelve 骨架可直接演变为：个人通讯录（name/address/birthday/phone/email）、员工薪资系统（涨薪+审批流）、配置管理（shelve 存程序配置）、小型 CRM。大型项目可无缝升级到 sqlite3（SQL 查询、事务）、JSON（跨语言交换）、ORM（如 SQLAlchemy/Django ORM）。注意 pickle 有安全性问题——绝不要反序列化不可信来源的数据（可能执行任意代码），跨网络传输应改用 JSON。

·与其他语言（Java/C++）的区别：

主题	Python	Java/C++
实例属性声明	无需声明，self.name = name 即创建	必须显式声明字段类型
调用父类方法	Person.giveRaise(self, ...) 或 super()	super.giveRaise(...)
多态	运行时动态查找（鸭子类型）	编译期绑定 + 虚函数
封装	约定（下划线）而非强制私有	private/protected 关键字
运算符重载	内置操作统一走 xxx 钩子	仅 operator 重载，范围窄
构造函数	init（初始化器，非真构造）	与类同名，可重载多个
类/模块关系	类是模块命名空间里的名字，按需导入	类必须独立成文件，靠包管理

· Python 发展历史背景：shelve/dbm 是 Python 早期（1.x 时代）就有的持久化方案，源自 UNIX dbm 库；pickle 协议至今已演进到第 5 版（Python 3.8+）。本书此版（第 6 版，2022 年前后）改用 f-string（3.6+）做格式化，取代旧版章节里的 % 与 .format()。name =='main' 模式自 Python 诞生起就是模块双用途的基石，如今仍是 CLI 工具与库共存的标配写法。

·高级开发者需要掌握的相关知识：MRO 与 super() 的协作式多继承（第 32 章）、slots 内存优化、元类与类装饰器（第 39 章）、getattribute 与描述符协议、dataclasses（3.7+，可大幅简化 init/repr 样板）、pickle 的协议选择与安全性、对象池与缓存设计。

学习建议（Learning Advice）

·重要程度：★★★★★（5/5）。这是全书唯一一章"用完整案例串起 OOP 全部基础"，是第 29~31 章语法深挖前的必经之路；不会本章，后续类机制全部悬空。

·应该掌握到什么程度：

·能手写每一步的演进（person1 → person14），解释每步动机；

·能脱口说出：init 与 self 的作用、repr/str 区别、继承查找"最低层胜出"、Person.giveRaise(self, ...) 与 self.giveRaise() 的区别、dict/class/name/bases 的用途、shelve 的读写模式；

·能独立完成两个变形：把 Manager 改成"构造函数+方法都定制"的其他职业类；用本骨架做一个通讯录（含 shelve 存取）。

·后续应该学习哪些相关内容：

·立即：第 29 章（class 语法细节）、第 30 章（运算符重载全集）、第 31 章（设计模式与类组合）；

·中期：第 32 章（super 与多继承）、第 39 章（装饰器——回顾 @rangetest 预告）；

·实践：用 dataclasses + sqlite3 把本章骨架升级成现代版本，体会"样板代码被语言特性取代"的过程。

（本章完。英文原文逐段保留，中文翻译、深度理解、代码分析、测验与答案均完整收录。）